TUGAS BESAR

Teori Bahasa Formal dan Otomata

13524007 Muhammad Ashkar
13524039 Rhenaldy Cahyadi Putra
13524069 Miguel Rangga Deardo Sinaga
13524097 Muhammad Faiz Alfada Dharma

DAFTAR ISI

BAB I TEORI SINGKAT **3**
1.1 Syntax Analysis 3
1.2 Grammar 4
1.3 Parse Tree 5
1.4 Recursive Descent Parser 5
BAB II PERANCANGAN & IMPLEMENTASI **6**
2.1 Perancangan Program 6
2.1.1 Teknologi yang Digunakan 6
2.1.2 Struktur Program 6
2.1.3 Fungsi/Kelas Utama 7
2.1.4 Alur Kerja Program 9
2.1.5 Justifikasi Pilihan Implementasi 9
2.2 Hasil Implementasi 9
BAB III PENGUJIAN **12**
BAB IV ATURAN GRAMMAR **48**
BAB V KESIMPULAN & SARAN **57**
5.1 Kesimpulan 57
5.2 Saran 57
LAMPIRAN **58**
REFERENSI **58**

BAB I TEORI SINGKAT

1.1 Syntax Analysis

Analisis sintaks (syntax analysis) merupakan tahap pada proses kompilasi yang dilakukan oleh parser. Tahap ini bertugas memeriksa apakah urutan token hasil lexical analysis telah sesuai dengan aturan grammar bahasa pemrograman. Pada tahap sebelumnya, source code telah diubah menjadi token-token seperti identifier, keyword, operator, maupun simbol khusus. Token-token tersebut kemudian diproses oleh parser untuk menentukan apakah struktur program valid secara sintaksis. Tujuan utama analisis sintaks antara lain:

1. memverifikasi struktur program,
2. mendeteksi kesalahan sintaks (syntax error),
3. membangun parse tree untuk tahap kompilasi selanjutnya.

Parser melakukan pemeriksaan berdasarkan aturan produksi bahasa pemrograman. Jika urutan token tidak sesuai dengan grammar, maka parser akan menghasilkan syntax error. Sebagai contoh, grammar Arion untuk bagian $\langle program - header \rangle$ didefinisikan sebagai:

$$\langle program - header \rangle \rightarrow programsy \ + \ ident \ + \ semicolon$$

Artinya, setelah keyword program dan nama program (ident), parser mengharapkan token semicolon. Misalnya diberikan input token berikut:

```
programsy
ident(Hello)
period
```

Urutan token tersebut tidak valid karena token terakhir seharusnya semicolon, bukan period. Oleh karena itu, parser akan menghasilkan syntax error. Pada tugas besar ini, parser digunakan untuk membangun Parse Tree berdasarkan grammar bahasa Arion menggunakan metode *Recursive Descent Parsing.*

1.2 Grammar

Grammar merupakan sekumpulan aturan formal yang mendefinisikan struktur valid suatu bahasa pemrograman. Grammar menjadi dasar utama parser dalam menentukan apakah suatu program valid secara sintaksis. Grammar didefinisikan sebagai sebuah sistem yang terdiri atas:

$$G = (V, T, P, S)$$

dengan:

- V = himpunan simbol non-terminal (variable), yaitu simbol yang digunakan untuk membentuk struktur sintaksis program dan masih dapat dikembangkan menggunakan aturan produksi.
- T = himpunan simbol terminal, yaitu simbol akhir yang muncul langsung pada input dan tidak dapat dikembangkan lagi.
- P = himpunan aturan produksi (production rules) yang mendefinisikan bagaimana non-terminal dapat dikembangkan menjadi kombinasi terminal maupun non-terminal lainnya.
- S = simbol awal (start symbol), yaitu titik awal proses derivasi grammar.

Aturan grammar dituliskan menggunakan production rule dengan bentuk umum:

$$A \rightarrow \alpha$$

dengan A merupakan simbol non-terminal dan α merupakan kombinasi simbol terminal dan/atau non-terminal. Sebagai contoh, pada grammar bahasa Arion terdapat aturan:

$$\langle program \rangle \rightarrow \langle program - header \rangle + \langle declaration - part \rangle + \langle compound - statement \rangle + period$$

Aturan tersebut menyatakan bahwa sebuah <program> tersusun atas beberapa bagian yaitu program-header, declaration-part, compound-statement, dan token period. Grammar juga dapat dituliskan dengan menggunakan simbol tambahan untuk mempermudah pembacaan aturan, seperti:

- simbol | untuk menyatakan alternatif,
- simbol * untuk menyatakan pengulangan nol atau lebih,
- simbol ? untuk menyatakan elemen opsional.

Sebagai contoh, pada grammar bahasa Arion terdapat aturan:

$$\langle index - list \rangle \rightarrow (intcon \mid charcon \mid ident) + (comma + \langle index - list \rangle)*$$

Aturan tersebut menyatakan bahwa <index-list> terdiri atas satu atau lebih nilai index yang dipisahkan oleh token comma. Nilai index dapat berupa integer constant (intcon), character constant (charcon), maupun identifier (ident).

1.3 Parse Tree

Parse Tree merupakan representasi berbentuk pohon yang menunjukkan bagaimana token-token pada program membentuk struktur grammar. Pada Parse Tree:

- node internal merepresentasikan simbol non-terminal,
- leaf node merepresentasikan simbol terminal atau token hasil lexer.

Sebagai contoh, potongan kode:

$$a := 5$$

dapat direpresentasikan sebagai:

```
<assignment-statement>
├── ident(a)
├── becomes
└── <expression>
    └── intcon(5)
```

Pada milestone ini, output utama parser adalah Parse Tree yang ditampilkan ke terminal dan disimpan ke file .txt.

1.4 Recursive Descent Parser

Recursive Descent Parser merupakan metode parsing top-down yang menggunakan sekumpulan fungsi rekursif untuk merepresentasikan non-terminal pada grammar. Metode ini disebut top-down karena proses parsing dimulai dari simbol awal (start symbol) kemudian diturunkan secara bertahap hingga menghasilkan terminal yang sesuai dengan token input. Pada Recursive Descent Parser, setiap non-terminal diimplementasikan sebagai sebuah fungsi parser. Sebagai contoh, production rule:

$$\langle program \rangle \rightarrow \langle program - header \rangle + \langle declaration - part \rangle + \langle compound - statement \rangle + period$$

dapat diimplementasikan sebagai fungsi parser yang memanggil fungsi-fungsi non-terminal lainnya secara rekursif.

Parser membaca token secara berurutan dari kiri ke kanan (left-to-right scanning) menggunakan token aktif (lookahead token) untuk menentukan aturan grammar yang harus diproses berikutnya. Pada setiap proses pencocokan token, parser akan memeriksa apakah token saat ini sesuai dengan token yang diharapkan berdasarkan production rule grammar. Jika token tidak sesuai, parser akan menghasilkan ParsingError.

Metode Recursive Descent memiliki beberapa keunggulan, antara lain:

1. implementasi relatif sederhana,
2. struktur parser mengikuti grammar,
3. mudah dipahami dan dikembangkan,
4. mempermudah proses error handling.

Namun metode ini juga memiliki beberapa keterbatasan, yaitu:

1. grammar tidak boleh memiliki left recursion,
2. beberapa grammar perlu dimodifikasi agar sesuai untuk parsing *top-down*,
3. pemilihan production rule tertentu terkadang memerlukan lookahead token.

BAB II PERANCANGAN & IMPLEMENTASI

2.1 Perancangan Program

2.1.1 Teknologi yang Digunakan

Implementasi syntax analyzer ini dikembangkan menggunakan bahasa pemrograman C++ sesuai dengan spesifikasi tugas besar dan sebagai lanjutan dari milestone lexical analyzer sebelumnya. Dalam pengembangannya, program memanfaatkan Standard Template Library (STL) vector untuk menyimpan token hasil lexer serta child node pada struktur Parse Tree. Arsitektur program dirancang dengan pemisahan komponen melalui beberapa kelas utama, seperti Parser untuk proses *Recursive Descent Parsing*, ParseNode untuk merepresentasikan node pada Parse Tree, TreePrinter untuk menampilkan struktur tree secara hierarkis, serta ParsingError untuk menangani kesalahan sintaks melalui exception handling dengan pesan error yang informatif.

Setiap aturan grammar bahasa Arion diimplementasikan sebagai method tersendiri pada kelas Parser sehingga implementasi menjadi terstruktur, mudah dikembangkan, dan secara langsung merepresentasikan spesifikasi grammar yang diberikan. Pemisahan kode ke dalam beberapa header dan source file seperti token.hpp, node.hpp, dan treeprinter.hpp juga mendukung modularitas serta mempermudah pemeliharaan program.

2.1.2 Struktur Program

```
\---JKW-Tubes-IF2224-2026
    |   .gitignore
    |   main.exe
    |   Makefile
    |   README.md
    |
    +---doc
    |       .gitkeep
    |       Laporan-1-JKW.pdf
    |       Laporan-2-JKW.pdf
    |
    +---src
    |   |   main.cpp
    |   |
    |   +---lexer
    |   |       lexer.cpp
    |   |       lexer.hpp
    |   |       reader.cpp
    |   |       reader.hpp
    |   |       token.hpp
    |   |
    |   +---parser
    |   |       node.hpp
    |   |       parser.cpp
    |   |       parser.hpp
    |   |       treeprinter.cpp
    |   |       treeprinter.hpp
    |   |
    |   \---utils
    |           exception.hpp
    |           utils.cpp
    |           utils.hpp
    |
    \---test
        +---milestone-1
        |
        +---milestone-2
        |
        +---milestone-3
        |
        \---milestone-4
```

2.1.3 Fungsi/Kelas Utama

Program dibagi menjadi beberapa modul utama yang terletak di dalam direktori src.

1. Main Program

Main program berperan sebagai penghubung utama seluruh sistem dengan mengatur alur eksekusi program dari awal hingga akhir. Bagian utama program diawali dengan meminta input berupa path dari pengguna, kemudian menggunakan modul reader untuk membuka file tersebut. Selanjutnya, program menjalankan proses lexical analysis untuk menghasilkan daftar token yang sesuai dengan spesifikasi bahasa Arion. Token-token hasil lexer kemudian diproses oleh parser untuk dilakukan analisis sintaks serta pembentukan Parse Tree. Setelah proses parsing selesai, main program menangani keluaran hasil yang ditampilkan ke terminal.

2. Lexer
   Modul lexer merupakan hasil pengembangan dari milestone sebelumnya yang bertugas melakukan analisis leksikal terhadap source code. Komponen ini membaca input melalui modul reader, kemudian memproses karakter secara berurutan menggunakan pendekatan Deterministic Finite Automata (DFA) untuk mengelompokkan rangkaian karakter menjadi token-token yang sesuai dengan spesifikasi bahasa Arion. Hasil tokenisasi ini menjadi masukan utama bagi parser, sehingga lexer berperan penting dalam memastikan struktur input telah direpresentasikan dalam bentuk token yang dapat dianalisis lebih lanjut pada tahap syntax analysis.

3. Parser
   Modul parser merupakan modul yang diimplementasi pada milestone kali ini. Diimplementasikan menggunakan metode Recursive Descent Parsing, modul ini terdiri dari kelas Parser yang berisi kumpulan fungsi grammar seperti program(), declarationPart(), statement(), expression(), dan fungsi non-terminal lainnya sesuai spesifikasi bahasa Arion. Parser menerima vector token dari lexer, melakukan validasi sintaks berdasarkan grammar yang di-hardcode, serta membangun Parse Tree melalui node-node yang dihasilkan selama proses parsing menggunakan struktur ParseNode, kelas yang merepresentasikan setiap simpul dalam pohon dengan label terminal atau non-terminal beserta child node untuk membentuk hubungan hierarkis sesuai grammar.

   Selain itu, modul parser juga didukung oleh TreePrinter, yang bertugas menampilkan Parse Tree secara hierarkis ke keluaran terminal dengan format tree terstruktur menggunakan traversal rekursif pada setiap node. Melalui proses ini, hubungan parent-child antar simpul dapat divisualisasikan dengan jelas sehingga memudahkan proses debugging dan validasi hasil parsing.

4. Exception (Utils)

Komponen ini digunakan untuk penanganan kesalahan sintaks selama proses parsing. ParsingError merupakan custom exception yang memberikan pesan error informatif ketika token yang ditemukan tidak sesuai dengan token yang diharapkan berdasarkan grammar. Pendekatan ini membantu program mendeteksi syntax error tanpa crash dan mempermudah evaluasi kesalahan input.

2.1.4 Alur Kerja Program

Alur kerja program secara umum adalah sebagai berikut:

1. Program utama (main.cpp) menerima path file kode sumber sebagai argumen.
2. Reader membaca isi file tersebut.
3. Lexer memproses teks dari Reader dan menghasilkan sebuah daftar token.
4. Parser menerima daftar token tersebut dan memulai proses parsing dari simbol awal grammar.
5. Selama proses parsing, Parser akan memanggil fungsi-fungsi rekursif yang merepresentasikan non-terminal, mencocokkan token yang diharapkan, dan membangun ParseNode secara hierarkis.
6. Jika seluruh token berhasil diproses sesuai grammar, TreePrinter akan mencetak Parse Tree yang telah terbentuk.
7. Jika terjadi ketidaksesuaian token dengan grammar, ParsingError akan dilemparkan dan ditangkap untuk menampilkan pesan kesalahan yang informatif.

2.1.5 Justifikasi Pilihan Implementasi

Implementasi program disusun dengan menyesuaikan kebutuhan spesifikasi milestone ini, yaitu melanjutkan hasil lexical analyzer ke tahap syntax analyzer yang mampu memvalidasi grammar bahasa Arion sekaligus membangun Parse Tree. Struktur parser dibuat dengan mengimplementasikan setiap aturan grammar ke dalam fungsi-fungsi parser secara terpisah, sehingga setiap bagian sintaks seperti declaration, statement, dan expression dapat diproses sesuai aturan yang telah ditentukan serta keseluruhan alur parsing menjadi lebih terstruktur dan mudah dibaca. Pembagian komponen seperti Lexer, Parser, ParseNode, TreePrinter, dan ParsingError juga menjaga struktur program tetap modular, sehingga implementasi lebih terorganisasi, mudah diuji, serta sesuai untuk mendukung kebutuhan milestone saat ini maupun pengembangan tahap berikutnya.

2.2 Hasil Implementasi

1. ParseNode

ParseNode adalah class yang digunakan untuk merepresentasikan sebuah node dalam parse tree. ParseNode digunakan untuk membentuk struktur pohon parser yang menyimpan nama node nya dan children nya. Setiap kali proses parsing berjalan dengan recursive descent parser, parser akan membentuk node baru sesuai aturan grammar yang sedang diproses dan dihubungkan dengan addChild().

```
class ParseNode {
public:
    string label;
    vector<ParseNode*> children;

    ParseNode(string lbl) {
        label = lbl;
    }

    void addChild(ParseNode* child) {
        children.push_back(child);
    }
};
```

2. match()
   Fungsi match mengecek apakah token saat ini sesuai dengan token yang diharapkan. Jika cocok, fungsi akan membuat ParseNode, lanjut membaca token selanjutnya dengan advance() lalu mengembalikan node tersebut. Jika tidak cocok, fungsi akan mengembalikan error.

```
ParseNode* Parser::match(TokenType expectedToken) {
    if(currToken == expectedToken) {
        string label = parserTokens[pos].str_type();

        if(currLexeme != "") {
            label += "(" + currLexeme + ")";
        }

        ParseNode* node = new ParseNode(label);
        advance();
        return node;
    }

    return error(expectedToken, currToken);
}
```

```
ParseNode* Parser::match(std::vector<TokenType> expectedTokens){
    for (TokenType expectedToken : expectedTokens){
        if(currToken == expectedToken) {
            string label = parserTokens[pos].str_type();

            if(currLexeme != "") {
                label += "(" + currLexeme + ")";
            }

            ParseNode* node = new ParseNode(label);
            advance();
            return node;
        }
    }

    return error(expectedTokens, currToken);
}
```

3. advance()
   Fungsi advance() digunakan untuk maju ke token berikutnya saat proses parsing. Jika posisinya masih valid, curToken dan curLexeme akan diperbarui.

```
void Parser::advance() {
    pos++;
    if(pos < (int)parserTokens.size()) {
        currToken = parserTokens[pos].type;
        currLexeme = parserTokens[pos].lexeme;
    }
}
```

BAB III PENGUJIAN

Masukkan (Input)

```
program Hello;

var
  a, b: integer;

begin
  a := 5;
  b := a + 10;
  writeln('Result = ', b);
end.
```

Keluaran (output)

```
Input path file: test/milestone-2/test1.txt
\-- <program>
    |-- <program-header>
    |   |-- PROGRAMSY
    |   |-- IDENT(Hello)
    |   \-- SEMICOLON
    |-- <declaration-part>
    |   \-- <var-declaration>
    |       |-- VARSY
    |       |-- <identifier-list>
    |       |   |-- IDENT(a)
    |       |   |-- COMMA
    |       |   \-- IDENT(b)
    |       |-- COLON
    |       |-- <type>
    |       |   \-- IDENT(integer)
    |       \-- SEMICOLON
    |-- <compound-statement>
    |   |-- BEGINSY
    |   |-- <statement-list>
    |   |   |-- <statement>
    |   |   |   \-- <assignment-statement>
    |   |   |       |-- <variable>
    |   |   |       |   \-- IDENT(a)
    |   |   |       |-- BECOMES
    |   |   |       \-- <expression>
    |   |   |           \-- <simple-expression>
    |   |   |               \-- <term>
    |   |   |                   \-- <factor>
    |   |   |                       \-- INTCON(5)
    |   |   |-- SEMICOLON
    |   |   |-- <statement>
    |   |   |   \-- <assignment-statement>
    |   |   |       |-- <variable>
```

```
    |   |   |       |   \-- IDENT(b)
    |   |   |       |-- BECOMES
    |   |   |       \-- <expression>
    |   |   |           \-- <simple-expression>
    |   |   |               |-- <term>
    |   |   |               |   \-- <factor>
    |   |   |               |       \-- IDENT(a)
    |   |   |               |-- <additive-operator>
    |   |   |               |   \-- PLUS
    |   |   |               \-- <term>
    |   |   |                   \-- <factor>
    |   |   |                       \-- INTCON(10)
    |   |   |-- SEMICOLON
    |   |   |-- <statement>
    |   |   |   \-- <procedure/function-call>
    |   |   |       |-- IDENT(writeln)
    |   |   |       |-- LPARENT
    |   |   |       |-- <parameter-list>
    |   |   |       |   |-- <expression>
    |   |   |       |   |   \-- <simple-expression>
    |   |   |       |   |       \-- <term>
    |   |   |       |   |           \-- <factor>
    |   |   |       |   |               \-- STRING('Result = ')
    |   |   |       |   |-- COMMA
    |   |   |       |   \-- <expression>
    |   |   |       |       \-- <simple-expression>
    |   |   |       |           \-- <term>
    |   |   |       |               \-- <factor>
    |   |   |       |                   \-- IDENT(b)
    |   |   |       \-- RPARENT
    |   |   \-- SEMICOLON
    |   \-- ENDSY
    \-- PERIOD
```

Masukkan (Input)

```
program TestControlFlow;

var
  a, b, c: integer;
  x: real;

begin
  a := 10;
  b := 3;
  x := 3.14;

  c := a + b * 2 - 1;
  x := x / 2.0;

  if a > b then
```

```
    writeln('a lebih besar')
  else
    writeln('b lebih besar atau sama');

  if (a == 10) and (b <> 0) then
    writeln('valid')
  else if (a < 0) or (b <= 0) then
    writeln('negatif');

  while a > 0 do
    a := a - 1;

  repeat
    b := b + 1
  until b >= 10;

  for c := 1 to 5 do
    writeln(c);

  for c := 5 downto 1 do
    writeln(c);

  writeln('selesai', c, x)
end.
```

Keluaran (output)

```
Input path file: test/milestone-2/test2.txt
\-- <program>
    |-- <program-header>
    |   |-- PROGRAMSY
    |   |-- IDENT(TestControlFlow)
    |   \-- SEMICOLON
    |-- <declaration-part>
    |   \-- <var-declaration>
    |       |-- VARSY
    |       |-- <identifier-list>
    |       |   |-- IDENT(a)
    |       |   |-- COMMA
    |       |   |-- IDENT(b)
    |       |   |-- COMMA
    |       |   \-- IDENT(c)
    |       |-- COLON
    |       |-- <type>
    |       |   \-- IDENT(integer)
    |       |-- SEMICOLON
    |       |-- <identifier-list>
    |       |   \-- IDENT(x)
    |       |-- COLON
    |       |-- <type>
    |       |   \-- IDENT(real)
    |       \-- SEMICOLON
    |-- <compound-statement>
    |   |-- BEGINSY
    |   |-- <statement-list>
    |   |   |-- <statement>
    |   |   |   \-- <assignment-statement>
```

```
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(a)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- INTCON(10)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(b)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- INTCON(3)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(x)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- REALCON(3.14)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(c)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               |-- <term>
|   |   |               |   \-- <factor>
|   |   |               |       \-- IDENT(a)
|   |   |               |-- <additive-operator>
|   |   |               |   \-- PLUS
|   |   |               |-- <term>
|   |   |               |   |-- <factor>
|   |   |               |   |   \-- IDENT(b)
|   |   |               |   |-- <multiplicative-operator>
|   |   |               |   |   \-- TIMES
|   |   |               |   \-- <factor>
|   |   |               |       \-- INTCON(2)
|   |   |               |-- <additive-operator>
|   |   |               |   \-- MINUS
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- INTCON(1)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(x)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   |-- <factor>
|   |   |                   |   \-- IDENT(x)
|   |   |                   |-- <multiplicative-operator>
|   |   |                   |   \-- RDIV
|   |   |                   \-- <factor>
|   |   |                       \-- REALCON(2.0)
```

```
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <if-statement>
|   |   |       |-- IFSY
|   |   |       |-- <expression>
|   |   |       |   |-- <simple-expression>
|   |   |       |   |   \-- <term>
|   |   |       |   |       \-- <factor>
|   |   |       |   |           \-- IDENT(a)
|   |   |       |   |-- <relational-operator>
|   |   |       |   |   \-- GTR
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               \-- IDENT(b)
|   |   |       |-- THENSY
|   |   |       |-- <statement>
|   |   |       |   \-- <procedure/function-call>
|   |   |       |       |-- IDENT(writeln)
|   |   |       |       |-- LPARENT
|   |   |       |       |-- <parameter-list>
|   |   |       |       |   \-- <expression>
|   |   |       |       |       \-- <simple-expression>
|   |   |       |       |           \-- <term>
|   |   |       |       |               \-- <factor>
|   |   |       |       |                   \-- STRING('a lebih besar')
|   |   |       |       \-- RPARENT
|   |   |       |-- ELSESY
|   |   |       \-- <statement>
|   |   |           \-- <procedure/function-call>
|   |   |               |-- IDENT(writeln)
|   |   |               |-- LPARENT
|   |   |               |-- <parameter-list>
|   |   |               |   \-- <expression>
|   |   |               |       \-- <simple-expression>
|   |   |               |           \-- <term>
|   |   |               |               \-- <factor>
|   |   |               |                   \-- STRING('b lebih besar atau sama')
|   |   |               \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <if-statement>
|   |   |       |-- IFSY
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           |-- <factor>
|   |   |       |           |   |-- LPARENT
|   |   |       |           |   |-- <expression>
|   |   |       |           |   |   |-- <simple-expression>
|   |   |       |           |   |   |   \-- <term>
|   |   |       |           |   |   |       \-- <factor>
|   |   |       |           |   |   |           \-- IDENT(a)
|   |   |       |           |   |   |-- <relational-operator>
|   |   |       |           |   |   |   \-- EQL
|   |   |       |           |   |   \-- <simple-expression>
|   |   |       |           |   |       \-- <term>
|   |   |       |           |   |           \-- <factor>
|   |   |       |           |   |               \-- INTCON(10)
|   |   |       |           |   \-- RPARENT
|   |   |       |           |-- <multiplicative-operator>
|   |   |       |           |   \-- ANDSY
|   |   |       |           \-- <factor>
|   |   |       |               |-- LPARENT
|   |   |       |               |-- <expression>
|   |   |       |               |   |-- <simple-expression>
|   |   |       |               |   |   \-- <term>
|   |   |       |               |   |       \-- <factor>
|   |   |       |               |   |           \-- IDENT(b)
|   |   |       |               |   |-- <relational-operator>
```

```
|   |   |       |               |   |   \-- NEQ
|   |   |       |               |   \-- <simple-expression>
|   |   |       |               |       \-- <term>
|   |   |       |               |           \-- <factor>
|   |   |       |               |               \-- INTCON(0)
|   |   |       |               \-- RPARENT
|   |   |       |-- THENSY
|   |   |       |-- <statement>
|   |   |       |   \-- <procedure/function-call>
|   |   |       |       |-- IDENT(writeln)
|   |   |       |       |-- LPARENT
|   |   |       |       |-- <parameter-list>
|   |   |       |       |   \-- <expression>
|   |   |       |       |       \-- <simple-expression>
|   |   |       |       |           \-- <term>
|   |   |       |       |               \-- <factor>
|   |   |       |       |                   \-- STRING('valid')
|   |   |       |       \-- RPARENT
|   |   |       |-- ELSESY
|   |   |       \-- <statement>
|   |   |           \-- <if-statement>
|   |   |               |-- IFSY
|   |   |               |-- <expression>
|   |   |               |   \-- <simple-expression>
|   |   |               |       |-- <term>
|   |   |               |       |   \-- <factor>
|   |   |               |       |       |-- LPARENT
|   |   |               |       |       |-- <expression>
|   |   |               |       |       |   |-- <simple-expression>
|   |   |               |       |       |   |   \-- <term>
|   |   |               |       |       |   |       \-- <factor>
|   |   |               |       |       |   |           \-- IDENT(a)
|   |   |               |       |       |   |-- <relational-operator>
|   |   |               |       |       |   |   \-- LSS
|   |   |               |       |       |   \-- <simple-expression>
|   |   |               |       |       |       \-- <term>
|   |   |               |       |       |           \-- <factor>
|   |   |               |       |       |               \-- INTCON(0)
|   |   |               |       |       \-- RPARENT
|   |   |               |       |-- <additive-operator>
|   |   |               |       |   \-- ORSY
|   |   |               |       \-- <term>
|   |   |               |           \-- <factor>
|   |   |               |               |-- LPARENT
|   |   |               |               |-- <expression>
|   |   |               |               |   |-- <simple-expression>
|   |   |               |               |   |   \-- <term>
|   |   |               |               |   |       \-- <factor>
|   |   |               |               |   |           \-- IDENT(b)
|   |   |               |               |   |-- <relational-operator>
|   |   |               |               |   |   \-- LEQ
|   |   |               |               |   \-- <simple-expression>
|   |   |               |               |       \-- <term>
|   |   |               |               |           \-- <factor>
|   |   |               |               |               \-- INTCON(0)
|   |   |               |               \-- RPARENT
|   |   |               |-- THENSY
|   |   |               \-- <statement>
|   |   |                   \-- <procedure/function-call>
|   |   |                       |-- IDENT(writeln)
|   |   |                       |-- LPARENT
|   |   |                       |-- <parameter-list>
|   |   |                       |   \-- <expression>
|   |   |                       |       \-- <simple-expression>
|   |   |                       |           \-- <term>
|   |   |                       |               \-- <factor>
|   |   |                       |                   \-- STRING('negatif')
|   |   |                       \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- <statement>
```

```
|   |   |   \-- <while-statement>
|   |   |       |-- WHILESY
|   |   |       |-- <expression>
|   |   |       |   |-- <simple-expression>
|   |   |       |   |   \-- <term>
|   |   |       |   |       \-- <factor>
|   |   |       |   |           \-- IDENT(a)
|   |   |       |   |-- <relational-operator>
|   |   |       |   |   \-- GTR
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               \-- INTCON(0)
|   |   |       |-- DOSY
|   |   |       \-- <statement>
|   |   |           \-- <assignment-statement>
|   |   |               |-- <variable>
|   |   |               |   \-- IDENT(a)
|   |   |               |-- BECOMES
|   |   |               \-- <expression>
|   |   |                   \-- <simple-expression>
|   |   |                       |-- <term>
|   |   |                       |   \-- <factor>
|   |   |                       |       \-- IDENT(a)
|   |   |                       |-- <additive-operator>
|   |   |                       |   \-- MINUS
|   |   |                       \-- <term>
|   |   |                           \-- <factor>
|   |   |                               \-- INTCON(1)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <repeat-statement>
|   |   |       |-- REPEATSY
|   |   |       |-- <statement-list>
|   |   |       |   \-- <statement>
|   |   |       |       \-- <assignment-statement>
|   |   |       |           |-- <variable>
|   |   |       |           |   \-- IDENT(b)
|   |   |       |           |-- BECOMES
|   |   |       |           \-- <expression>
|   |   |       |               \-- <simple-expression>
|   |   |       |                   |-- <term>
|   |   |       |                   |   \-- <factor>
|   |   |       |                   |       \-- IDENT(b)
|   |   |       |                   |-- <additive-operator>
|   |   |       |                   |   \-- PLUS
|   |   |       |                   \-- <term>
|   |   |       |                       \-- <factor>
|   |   |       |                           \-- INTCON(1)
|   |   |       |-- UNTILSY
|   |   |       \-- <expression>
|   |   |           |-- <simple-expression>
|   |   |           |   \-- <term>
|   |   |           |       \-- <factor>
|   |   |           |           \-- IDENT(b)
|   |   |           |-- <relational-operator>
|   |   |           |   \-- GEQ
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- INTCON(10)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <for-statement>
|   |   |       |-- FORSY
|   |   |       |-- IDENT(c)
|   |   |       |-- BECOMES
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
```

```
|   |   |       |           \-- <factor>
|   |   |       |               \-- INTCON(1)
|   |   |       |-- TOSY
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               \-- INTCON(5)
|   |   |       |-- DOSY
|   |   |       \-- <statement>
|   |   |           \-- <procedure/function-call>
|   |   |               |-- IDENT(writeln)
|   |   |               |-- LPARENT
|   |   |               |-- <parameter-list>
|   |   |               |   \-- <expression>
|   |   |               |       \-- <simple-expression>
|   |   |               |           \-- <term>
|   |   |               |               \-- <factor>
|   |   |               |                   \-- IDENT(c)
|   |   |               \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <for-statement>
|   |   |       |-- FORSY
|   |   |       |-- IDENT(c)
|   |   |       |-- BECOMES
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               \-- INTCON(5)
|   |   |       |-- DOWNTOSY
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               \-- INTCON(1)
|   |   |       |-- DOSY
|   |   |       \-- <statement>
|   |   |           \-- <procedure/function-call>
|   |   |               |-- IDENT(writeln)
|   |   |               |-- LPARENT
|   |   |               |-- <parameter-list>
|   |   |               |   \-- <expression>
|   |   |               |       \-- <simple-expression>
|   |   |               |           \-- <term>
|   |   |               |               \-- <factor>
|   |   |               |                   \-- IDENT(c)
|   |   |               \-- RPARENT
|   |   |-- SEMICOLON
|   |   \-- <statement>
|   |       \-- <procedure/function-call>
|   |           |-- IDENT(writeln)
|   |           |-- LPARENT
|   |           |-- <parameter-list>
|   |           |   |-- <expression>
|   |           |   |   \-- <simple-expression>
|   |           |   |       \-- <term>
|   |           |   |           \-- <factor>
|   |           |   |               \-- STRING('selesai')
|   |           |   |-- COMMA
|   |           |   |-- <expression>
|   |           |   |   \-- <simple-expression>
|   |           |   |       \-- <term>
|   |           |   |           \-- <factor>
|   |           |   |               \-- IDENT(c)
|   |           |   |-- COMMA
|   |           |   \-- <expression>
|   |           |       \-- <simple-expression>
|   |           |           \-- <term>
```

```
    |   |           |               \-- <factor>
    |   |           |                   \-- IDENT(x)
    |   |           \-- RPARENT
    |   \-- ENDSY
    \-- PERIOD
```

Masukkan (Input)

```
program TestConstEnumRange;

const
  MAX == 100;
  MIN == -10;
  PI == 3.14;
  MSG == 'hello';
  CH == 'a';

type
  Direction == (north, south, east, west);
  Month == (jan, feb, mar, apr, may, jun);
  Letter == 'a'..'z';
  UpperLetter == 'A'..'Z';

var
  d: integer;
  i: integer;

begin
  d := 5;
  i := MAX;

  case d of
    0: writeln('nol');
    1, 2, 3: writeln('kecil');
    4, 5: writeln('sedang');
    6, 7, 8, 9: writeln('besar')
  end;

  case i of
    100: writeln('maksimum')
  end;
end.
```

Keluaran (output)

```
Input path file: test/milestone-2/test3.txt
\-- <program>
    |-- <program-header>
    |   |-- PROGRAMSY
```

```
|   |-- IDENT(TestConstEnumRange)
|   \-- SEMICOLON
|-- <declaration-part>
|   |-- <const-declaration>
|   |   |-- CONSTSY
|   |   |-- IDENT(MAX)
|   |   |-- EQL
|   |   |-- <constant>
|   |   |   \-- INTCON(100)
|   |   |-- SEMICOLON
|   |   |-- IDENT(MIN)
|   |   |-- EQL
|   |   |-- <constant>
|   |   |   \-- INTCON(10)
|   |   |-- SEMICOLON
|   |   |-- IDENT(PI)
|   |   |-- EQL
|   |   |-- <constant>
|   |   |   \-- REALCON(3.14)
|   |   |-- SEMICOLON
|   |   |-- IDENT(MSG)
|   |   |-- EQL
|   |   |-- <constant>
|   |   |   \-- STRING('hello')
|   |   |-- SEMICOLON
|   |   |-- IDENT(CH)
|   |   |-- EQL
|   |   |-- <constant>
|   |   |   \-- CHARCON('a')
|   |   \-- SEMICOLON
|   |-- <type-declaration>
|   |   |-- TYPESY
|   |   |-- IDENT(Direction)
|   |   |-- EQL
|   |   |-- <type>
|   |   |   \-- <enumerated>
|   |   |       |-- LPARENT
|   |   |       |-- IDENT(north)
|   |   |       |-- COMMA
|   |   |       |-- IDENT(south)
|   |   |       |-- COMMA
|   |   |       |-- IDENT(east)
|   |   |       |-- COMMA
|   |   |       |-- IDENT(west)
|   |   |       \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- IDENT(Month)
|   |   |-- EQL
|   |   |-- <type>
|   |   |   \-- <enumerated>
|   |   |       |-- LPARENT
|   |   |       |-- IDENT(jan)
|   |   |       |-- COMMA
|   |   |       |-- IDENT(feb)
|   |   |       |-- COMMA
|   |   |       |-- IDENT(mar)
|   |   |       |-- COMMA
|   |   |       |-- IDENT(apr)
|   |   |       |-- COMMA
|   |   |       |-- IDENT(may)
|   |   |       |-- COMMA
|   |   |       |-- IDENT(jun)
|   |   |       \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- IDENT(Letter)
|   |   |-- EQL
|   |   |-- <type>
|   |   |   \-- <range>
|   |   |       |-- <constant>
|   |   |       |   \-- CHARCON('a')
```

```
|   |   |       |-- PERIOD
|   |   |       |-- PERIOD
|   |   |       \-- <constant>
|   |   |           \-- CHARCON('z')
|   |   |-- SEMICOLON
|   |   |-- IDENT(UpperLetter)
|   |   |-- EQL
|   |   |-- <type>
|   |   |   \-- <range>
|   |   |       |-- <constant>
|   |   |       |   \-- CHARCON('A')
|   |   |       |-- PERIOD
|   |   |       |-- PERIOD
|   |   |       \-- <constant>
|   |   |           \-- CHARCON('Z')
|   |   \-- SEMICOLON
|   \-- <var-declaration>
|       |-- VARSY
|       |-- <identifier-list>
|       |   \-- IDENT(d)
|       |-- COLON
|       |-- <type>
|       |   \-- IDENT(integer)
|       |-- SEMICOLON
|       |-- <identifier-list>
|       |   \-- IDENT(i)
|       |-- COLON
|       |-- <type>
|       |   \-- IDENT(integer)
|       \-- SEMICOLON
|-- <compound-statement>
|   |-- BEGINSY
|   |-- <statement-list>
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(d)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- INTCON(5)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(i)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- IDENT(MAX)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <case-statement>
|   |   |       |-- CASESY
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               \-- IDENT(d)
|   |   |       |-- OFSY
|   |   |       |-- <case-block>
|   |   |       |   |-- <constant>
|   |   |       |   |   \-- INTCON(0)
|   |   |       |   |-- COLON
|   |   |       |   |-- <statement>
|   |   |       |   |   \-- <procedure/function-call>
```

```
|   |   |       |   |       |-- IDENT(writeln)
|   |   |       |   |       |-- LPARENT
|   |   |       |   |       |-- <parameter-list>
|   |   |       |   |       |   \-- <expression>
|   |   |       |   |       |       \-- <simple-expression>
|   |   |       |   |       |           \-- <term>
|   |   |       |   |       |               \-- <factor>
|   |   |       |   |       |                   \-- STRING('nol')
|   |   |       |   |       \-- RPARENT
|   |   |       |   |-- SEMICOLON
|   |   |       |   \-- <case-block>
|   |   |       |       |-- <constant>
|   |   |       |       |   \-- INTCON(1)
|   |   |       |       |-- COMMA
|   |   |       |       |-- <constant>
|   |   |       |       |   \-- INTCON(2)
|   |   |       |       |-- COMMA
|   |   |       |       |-- <constant>
|   |   |       |       |   \-- INTCON(3)
|   |   |       |       |-- COLON
|   |   |       |       |-- <statement>
|   |   |       |       |   \-- <procedure/function-call>
|   |   |       |       |       |-- IDENT(writeln)
|   |   |       |       |       |-- LPARENT
|   |   |       |       |       |-- <parameter-list>
|   |   |       |       |       |   \-- <expression>
|   |   |       |       |       |       \-- <simple-expression>
|   |   |       |       |       |           \-- <term>
|   |   |       |       |       |               \-- <factor>
|   |   |       |       |       |                   \-- STRING('kecil')
|   |   |       |       |       \-- RPARENT
|   |   |       |       |-- SEMICOLON
|   |   |       |       \-- <case-block>
|   |   |       |           |-- <constant>
|   |   |       |           |   \-- INTCON(4)
|   |   |       |           |-- COMMA
|   |   |       |           |-- <constant>
|   |   |       |           |   \-- INTCON(5)
|   |   |       |           |-- COLON
|   |   |       |           |-- <statement>
|   |   |       |           |   \-- <procedure/function-call>
|   |   |       |           |       |-- IDENT(writeln)
|   |   |       |           |       |-- LPARENT
|   |   |       |           |       |-- <parameter-list>
|   |   |       |           |       |   \-- <expression>
|   |   |       |           |       |       \-- <simple-expression>
|   |   |       |           |       |           \-- <term>
|   |   |       |           |       |               \-- <factor>
|   |   |       |           |       |                   \-- STRING('sedang')
|   |   |       |           |       \-- RPARENT
|   |   |       |           |-- SEMICOLON
|   |   |       |           \-- <case-block>
|   |   |       |               |-- <constant>
|   |   |       |               |   \-- INTCON(6)
|   |   |       |               |-- COMMA
|   |   |       |               |-- <constant>
|   |   |       |               |   \-- INTCON(7)
|   |   |       |               |-- COMMA
|   |   |       |               |-- <constant>
|   |   |       |               |   \-- INTCON(8)
|   |   |       |               |-- COMMA
|   |   |       |               |-- <constant>
|   |   |       |               |   \-- INTCON(9)
|   |   |       |               |-- COLON
|   |   |       |               \-- <statement>
|   |   |       |                   \-- <procedure/function-call>
|   |   |       |                       |-- IDENT(writeln)
|   |   |       |                       |-- LPARENT
|   |   |       |                       |-- <parameter-list>
|   |   |       |                       |   \-- <expression>
```

```
    |   |   |       |                       |       \-- <simple-expression>
    |   |   |       |                       |           \-- <term>
    |   |   |       |                       |               \-- <factor>
    |   |   |       |                       |                   \-- STRING('besar')
    |   |   |       |                       \-- RPARENT
    |   |   |       \-- ENDSY
    |   |   |-- SEMICOLON
    |   |   |-- <statement>
    |   |   |   \-- <case-statement>
    |   |   |       |-- CASESY
    |   |   |       |-- <expression>
    |   |   |       |   \-- <simple-expression>
    |   |   |       |       \-- <term>
    |   |   |       |           \-- <factor>
    |   |   |       |               \-- IDENT(i)
    |   |   |       |-- OFSY
    |   |   |       |-- <case-block>
    |   |   |       |   |-- <constant>
    |   |   |       |   |   \-- INTCON(100)
    |   |   |       |   |-- COLON
    |   |   |       |   \-- <statement>
    |   |   |       |       \-- <procedure/function-call>
    |   |   |       |           |-- IDENT(writeln)
    |   |   |       |           |-- LPARENT
    |   |   |       |           |-- <parameter-list>
    |   |   |       |           |   \-- <expression>
    |   |   |       |           |       \-- <simple-expression>
    |   |   |       |           |           \-- <term>
    |   |   |       |           |               \-- <factor>
    |   |   |       |           |                   \-- STRING('maksimum')
    |   |   |       |           \-- RPARENT
    |   |   |       \-- ENDSY
    |   |   \-- SEMICOLON
    |   \-- ENDSY
    \-- PERIOD
```

Masukkan (Input)

```
program TestRecordArray;

type
  Point == record
    x, y: integer;
    label: string;
  end;
  Person == record
    name: string;
    age: integer;
    height: real;
  end;
  IntArray == array[1..10] of integer;
  Matrix == array[1..5] of real;
  CharMap == array[char] of integer;

var
  i: integer;
  x: real;
```

```
begin
  i := 0;
  x := 3;
  writeln(i);
  writeln(x)
end.
```

Keluaran (output)

```
Input path file: test/milestone-2/test4.txt
\-- <program>
    |-- <program-header>
    |   |-- PROGRAMSY
    |   |-- IDENT(TestRecordArray)
    |   \-- SEMICOLON
    |-- <declaration-part>
    |   |-- <type-declaration>
    |   |   |-- TYPESY
    |   |   |-- IDENT(Point)
    |   |   |-- EQL
    |   |   |-- <type>
    |   |   |   \-- <record-type>
    |   |   |       |-- RECORDSY
    |   |   |       |-- <field-list>
    |   |   |       |   |-- <field-part>
    |   |   |       |   |   |-- <identifier-list>
    |   |   |       |   |   |   |-- IDENT(x)
    |   |   |       |   |   |   |-- COMMA
    |   |   |       |   |   |   \-- IDENT(y)
    |   |   |       |   |   |-- COLON
    |   |   |       |   |   \-- <type>
    |   |   |       |   |       \-- IDENT(integer)
    |   |   |       |   |-- SEMICOLON
    |   |   |       |   |-- <field-part>
    |   |   |       |   |   |-- <identifier-list>
    |   |   |       |   |   |   \-- IDENT(label)
    |   |   |       |   |   |-- COLON
    |   |   |       |   |   \-- <type>
    |   |   |       |   |       \-- IDENT(string)
    |   |   |       |   \-- SEMICOLON
    |   |   |       \-- ENDSY
    |   |   |-- SEMICOLON
    |   |   |-- IDENT(Person)
    |   |   |-- EQL
    |   |   |-- <type>
    |   |   |   \-- <record-type>
    |   |   |       |-- RECORDSY
    |   |   |       |-- <field-list>
    |   |   |       |   |-- <field-part>
    |   |   |       |   |   |-- <identifier-list>
    |   |   |       |   |   |   \-- IDENT(name)
    |   |   |       |   |   |-- COLON
    |   |   |       |   |   \-- <type>
```

```
|   |   |       |   |       \-- IDENT(string)
|   |   |       |   |-- SEMICOLON
|   |   |       |   |-- <field-part>
|   |   |       |   |   |-- <identifier-list>
|   |   |       |   |   |   \-- IDENT(age)
|   |   |       |   |   |-- COLON
|   |   |       |   |   \-- <type>
|   |   |       |   |       \-- IDENT(integer)
|   |   |       |   |-- SEMICOLON
|   |   |       |   |-- <field-part>
|   |   |       |   |   |-- <identifier-list>
|   |   |       |   |   |   \-- IDENT(height)
|   |   |       |   |   |-- COLON
|   |   |       |   |   \-- <type>
|   |   |       |   |       \-- IDENT(real)
|   |   |       |   \-- SEMICOLON
|   |   |       \-- ENDSY
|   |   |-- SEMICOLON
|   |   |-- IDENT(IntArray)
|   |   |-- EQL
|   |   |-- <type>
|   |   |   \-- <array-type>
|   |   |       |-- ARRAYSY
|   |   |       |-- LBRACK
|   |   |       |-- <range>
|   |   |       |   |-- <constant>
|   |   |       |   |   \-- INTCON(1)
|   |   |       |   |-- PERIOD
|   |   |       |   |-- PERIOD
|   |   |       |   \-- <constant>
|   |   |       |       \-- INTCON(10)
|   |   |       |-- RBRACK
|   |   |       |-- OFSY
|   |   |       \-- <type>
|   |   |           \-- IDENT(integer)
|   |   |-- SEMICOLON
|   |   |-- IDENT(Matrix)
|   |   |-- EQL
|   |   |-- <type>
|   |   |   \-- <array-type>
|   |   |       |-- ARRAYSY
|   |   |       |-- LBRACK
|   |   |       |-- <range>
|   |   |       |   |-- <constant>
|   |   |       |   |   \-- INTCON(1)
|   |   |       |   |-- PERIOD
|   |   |       |   |-- PERIOD
|   |   |       |   \-- <constant>
|   |   |       |       \-- INTCON(5)
|   |   |       |-- RBRACK
|   |   |       |-- OFSY
|   |   |       \-- <type>
|   |   |           \-- IDENT(real)
|   |   |-- SEMICOLON
|   |   |-- IDENT(CharMap)
|   |   |-- EQL
```

```
|   |   |-- <type>
|   |   |   \-- <array-type>
|   |   |       |-- ARRAYSY
|   |   |       |-- LBRACK
|   |   |       |-- IDENT(char)
|   |   |       |-- RBRACK
|   |   |       |-- OFSY
|   |   |       \-- <type>
|   |   |           \-- IDENT(integer)
|   |   \-- SEMICOLON
|   \-- <var-declaration>
|       |-- VARSY
|       |-- <identifier-list>
|       |   \-- IDENT(i)
|       |-- COLON
|       |-- <type>
|       |   \-- IDENT(integer)
|       |-- SEMICOLON
|       |-- <identifier-list>
|       |   \-- IDENT(x)
|       |-- COLON
|       |-- <type>
|       |   \-- IDENT(real)
|       \-- SEMICOLON
|-- <compound-statement>
|   |-- BEGINSY
|   |-- <statement-list>
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(i)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- INTCON(0)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(x)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- INTCON(3)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <procedure/function-call>
|   |   |       |-- IDENT(writeln)
|   |   |       |-- LPARENT
|   |   |       |-- <parameter-list>
|   |   |       |   \-- <expression>
|   |   |       |       \-- <simple-expression>
```

```
    |   |   |       |           \-- <term>
    |   |   |       |               \-- <factor>
    |   |   |       |                   \-- IDENT(i)
    |   |   |       \-- RPARENT
    |   |   |-- SEMICOLON
    |   |   \-- <statement>
    |   |       \-- <procedure/function-call>
    |   |           |-- IDENT(writeln)
    |   |           |-- LPARENT
    |   |           |-- <parameter-list>
    |   |           |   \-- <expression>
    |   |           |       \-- <simple-expression>
    |   |           |           \-- <term>
    |   |           |               \-- <factor>
    |   |           |                   \-- IDENT(x)
    |   |           \-- RPARENT
    |   \-- ENDSY
    \-- PERIOD
```

Masukkan (Input)

```
program TestSubprogram;

var
  result: integer;

function Max(a, b: integer): integer;
begin
  if a > b then
    Max := a
  else
    Max := b;
end;

function InRange(val, lo, hi: integer): boolean;
begin
  InRange := (val >= lo) and (val <= hi);
end;

procedure PrintNum(n: integer);
begin
  writeln(n);
end;

procedure PrintPair(a, b: integer; msg: string);
begin
  writeln(msg);
  writeln(a);
  writeln(b);
```

```
end;

procedure Greet;
begin
  writeln('Hello');
end;

begin
  result := Max(15, 7);
  PrintNum(result);
  PrintPair(1, 2, 'pasangan:');
  Greet();

  if not (result > 100) then
    writeln('dalam range');

  result := Max(Max(1, 2), 3)
end.
```

Keluaran (output)

```
Input path file: test/milestone-2/test5.txt
\-- <program>
    |-- <program-header>
    |   |-- PROGRAMSY
    |   |-- IDENT(TestSubprogram)
    |   \-- SEMICOLON
    |-- <declaration-part>
    |   |-- <var-declaration>
    |   |   |-- VARSY
    |   |   |-- <identifier-list>
    |   |   |   \-- IDENT(result)
    |   |   |-- COLON
    |   |   |-- <type>
    |   |   |   \-- IDENT(integer)
    |   |   \-- SEMICOLON
    |   |-- <subprogram-declaration>
    |   |   \-- <function-declaration>
    |   |       |-- FUNCTIONSY
    |   |       |-- IDENT(Max)
    |   |       |-- <formal-parameter-list>
    |   |       |   |-- LPARENT
    |   |       |   |-- <parameter-group>
    |   |       |   |   |-- <identifier-list>
    |   |       |   |   |   |-- IDENT(a)
    |   |       |   |   |   |-- COMMA
    |   |       |   |   |   \-- IDENT(b)
    |   |       |   |   |-- COLON
    |   |       |   |   \-- IDENT(integer)
    |   |       |   \-- RPARENT
    |   |       |-- COLON
    |   |       |-- IDENT(integer)
    |   |       |-- SEMICOLON
    |   |       |-- <block>
    |   |       |   |-- <declaration-part>
    |   |       |   \-- <compound-statement>
    |   |       |       |-- BEGINSY
    |   |       |       |-- <statement-list>
    |   |       |       |   |-- <statement>
    |   |       |       |   |   \-- <if-statement>
```

```
|   |       |       |   |       |-- IFSY
|   |       |       |   |       |-- <expression>
|   |       |       |   |       |   |-- <simple-expression>
|   |       |       |   |       |   |   \-- <term>
|   |       |       |   |       |   |       \-- <factor>
|   |       |       |   |       |   |           \-- IDENT(a)
|   |       |       |   |       |   |-- <relational-operator>
|   |       |       |   |       |   |   \-- GTR
|   |       |       |   |       |   \-- <simple-expression>
|   |       |       |   |       |       \-- <term>
|   |       |       |   |       |           \-- <factor>
|   |       |       |   |       |               \-- IDENT(b)
|   |       |       |   |       |-- THENSY
|   |       |       |   |       |-- <statement>
|   |       |       |   |       |   \-- <assignment-statement>
|   |       |       |   |       |       |-- <variable>
|   |       |       |   |       |       |   \-- IDENT(Max)
|   |       |       |   |       |       |-- BECOMES
|   |       |       |   |       |       \-- <expression>
|   |       |       |   |       |           \-- <simple-expression>
|   |       |       |   |       |               \-- <term>
|   |       |       |   |       |                   \-- <factor>
|   |       |       |   |       |                       \-- IDENT(a)
|   |       |       |   |       |-- ELSESY
|   |       |       |   |       \-- <statement>
|   |       |       |   |           \-- <assignment-statement>
|   |       |       |   |               |-- <variable>
|   |       |       |   |               |   \-- IDENT(Max)
|   |       |       |   |               |-- BECOMES
|   |       |       |   |               \-- <expression>
|   |       |       |   |                   \-- <simple-expression>
|   |       |       |   |                       \-- <term>
|   |       |       |   |                           \-- <factor>
|   |       |       |   |                               \-- IDENT(b)
|   |       |       |   \-- SEMICOLON
|   |       |       \-- ENDSY
|   |       \-- SEMICOLON
|   |-- <subprogram-declaration>
|   |   \-- <function-declaration>
|   |       |-- FUNCTIONSY
|   |       |-- IDENT(InRange)
|   |       |-- <formal-parameter-list>
|   |       |   |-- LPARENT
|   |       |   |-- <parameter-group>
|   |       |   |   |-- <identifier-list>
|   |       |   |   |   |-- IDENT(val)
|   |       |   |   |   |-- COMMA
|   |       |   |   |   |-- IDENT(lo)
|   |       |   |   |   |-- COMMA
|   |       |   |   |   \-- IDENT(hi)
|   |       |   |   |-- COLON
|   |       |   |   \-- IDENT(integer)
|   |       |   \-- RPARENT
|   |       |-- COLON
|   |       |-- IDENT(boolean)
|   |       |-- SEMICOLON
|   |       |-- <block>
|   |       |   |-- <declaration-part>
|   |       |   \-- <compound-statement>
|   |       |       |-- BEGINSY
|   |       |       |-- <statement-list>
|   |       |       |   |-- <statement>
|   |       |       |   |   \-- <assignment-statement>
|   |       |       |   |       |-- <variable>
|   |       |       |   |       |   \-- IDENT(InRange)
|   |       |       |   |       |-- BECOMES
|   |       |       |   |       \-- <expression>
|   |       |       |   |           \-- <simple-expression>
|   |       |       |   |               \-- <term>
|   |       |       |   |                   |-- <factor>
|   |       |       |   |                   |   |-- LPARENT
|   |       |       |   |                   |   |-- <expression>
|   |       |       |   |                   |   |   |-- <simple-expression>
|   |       |       |   |                   |   |   |   \-- <term>
|   |       |       |   |                   |   |   |       \-- <factor>
|   |       |       |   |                   |   |   |           \-- IDENT(val)
|   |       |       |   |                   |   |   |-- <relational-operator>
|   |       |       |   |                   |   |   |   \-- GEQ
|   |       |       |   |                   |   |   \-- <simple-expression>
|   |       |       |   |                   |   |       \-- <term>
```

```
|   |       |       |   |                   |   |           \-- <factor>
|   |       |       |   |                   |   |               \-- IDENT(lo)
|   |       |       |   |                   |   \-- RPARENT
|   |       |       |   |                   |-- <multiplicative-operator>
|   |       |       |   |                   |   \-- ANDSY
|   |       |       |   |                   \-- <factor>
|   |       |       |   |                       |-- LPARENT
|   |       |       |   |                       |-- <expression>
|   |       |       |   |                       |   |-- <simple-expression>
|   |       |       |   |                       |   |   \-- <term>
|   |       |       |   |                       |   |       \-- <factor>
|   |       |       |   |                       |   |           \-- IDENT(val)
|   |       |       |   |                       |   |-- <relational-operator>
|   |       |       |   |                       |   |   \-- LEQ
|   |       |       |   |                       |   \-- <simple-expression>
|   |       |       |   |                       |       \-- <term>
|   |       |       |   |                       |           \-- <factor>
|   |       |       |   |                       |               \-- IDENT(hi)
|   |       |       |   |                       \-- RPARENT
|   |       |       |   \-- SEMICOLON
|   |       |       \-- ENDSY
|   |       \-- SEMICOLON
|   |-- <subprogram-declaration>
|   |   \-- <procedure-declaration>
|   |       |-- PROCEDURESY
|   |       |-- IDENT(PrintNum)
|   |       |-- <formal-parameter-list>
|   |       |   |-- LPARENT
|   |       |   |-- <parameter-group>
|   |       |   |   |-- <identifier-list>
|   |       |   |   |   \-- IDENT(n)
|   |       |   |   |-- COLON
|   |       |   |   \-- IDENT(integer)
|   |       |   \-- RPARENT
|   |       |-- SEMICOLON
|   |       |-- <block>
|   |       |   |-- <declaration-part>
|   |       |   \-- <compound-statement>
|   |       |       |-- BEGINSY
|   |       |       |-- <statement-list>
|   |       |       |   |-- <statement>
|   |       |       |   |   \-- <procedure/function-call>
|   |       |       |   |       |-- IDENT(writeln)
|   |       |       |   |       |-- LPARENT
|   |       |       |   |       |-- <parameter-list>
|   |       |       |   |       |   \-- <expression>
|   |       |       |   |       |       \-- <simple-expression>
|   |       |       |   |       |           \-- <term>
|   |       |       |   |       |               \-- <factor>
|   |       |       |   |       |                   \-- IDENT(n)
|   |       |       |   |       \-- RPARENT
|   |       |       |   \-- SEMICOLON
|   |       |       \-- ENDSY
|   |       \-- SEMICOLON
|   |-- <subprogram-declaration>
|   |   \-- <procedure-declaration>
|   |       |-- PROCEDURESY
|   |       |-- IDENT(PrintPair)
|   |       |-- <formal-parameter-list>
|   |       |   |-- LPARENT
|   |       |   |-- <parameter-group>
|   |       |   |   |-- <identifier-list>
|   |       |   |   |   |-- IDENT(a)
|   |       |   |   |   |-- COMMA
|   |       |   |   |   \-- IDENT(b)
|   |       |   |   |-- COLON
|   |       |   |   \-- IDENT(integer)
|   |       |   |-- SEMICOLON
|   |       |   |-- <parameter-group>
|   |       |   |   |-- <identifier-list>
|   |       |   |   |   \-- IDENT(msg)
|   |       |   |   |-- COLON
|   |       |   |   \-- IDENT(string)
|   |       |   \-- RPARENT
|   |       |-- SEMICOLON
|   |       |-- <block>
|   |       |   |-- <declaration-part>
|   |       |   \-- <compound-statement>
|   |       |       |-- BEGINSY
|   |       |       |-- <statement-list>
```

```
|   |       |       |   |-- <statement>
|   |       |       |   |   \-- <procedure/function-call>
|   |       |       |   |       |-- IDENT(writeln)
|   |       |       |   |       |-- LPARENT
|   |       |       |   |       |-- <parameter-list>
|   |       |       |   |       |   \-- <expression>
|   |       |       |   |       |       \-- <simple-expression>
|   |       |       |   |       |           \-- <term>
|   |       |       |   |       |               \-- <factor>
|   |       |       |   |       |                   \-- IDENT(msg)
|   |       |       |   |       \-- RPARENT
|   |       |       |   |-- SEMICOLON
|   |       |       |   |-- <statement>
|   |       |       |   |   \-- <procedure/function-call>
|   |       |       |   |       |-- IDENT(writeln)
|   |       |       |   |       |-- LPARENT
|   |       |       |   |       |-- <parameter-list>
|   |       |       |   |       |   \-- <expression>
|   |       |       |   |       |       \-- <simple-expression>
|   |       |       |   |       |           \-- <term>
|   |       |       |   |       |               \-- <factor>
|   |       |       |   |       |                   \-- IDENT(a)
|   |       |       |   |       \-- RPARENT
|   |       |       |   |-- SEMICOLON
|   |       |       |   |-- <statement>
|   |       |       |   |   \-- <procedure/function-call>
|   |       |       |   |       |-- IDENT(writeln)
|   |       |       |   |       |-- LPARENT
|   |       |       |   |       |-- <parameter-list>
|   |       |       |   |       |   \-- <expression>
|   |       |       |   |       |       \-- <simple-expression>
|   |       |       |   |       |           \-- <term>
|   |       |       |   |       |               \-- <factor>
|   |       |       |   |       |                   \-- IDENT(b)
|   |       |       |   |       \-- RPARENT
|   |       |       |   \-- SEMICOLON
|   |       |       \-- ENDSY
|   |       \-- SEMICOLON
|   \-- <subprogram-declaration>
|       \-- <procedure-declaration>
|           |-- PROCEDURESY
|           |-- IDENT(Greet)
|           |-- SEMICOLON
|           |-- <block>
|           |   |-- <declaration-part>
|           |   \-- <compound-statement>
|           |       |-- BEGINSY
|           |       |-- <statement-list>
|           |       |   |-- <statement>
|           |       |   |   \-- <procedure/function-call>
|           |       |   |       |-- IDENT(writeln)
|           |       |   |       |-- LPARENT
|           |       |   |       |-- <parameter-list>
|           |       |   |       |   \-- <expression>
|           |       |   |       |       \-- <simple-expression>
|           |       |   |       |           \-- <term>
|           |       |   |       |               \-- <factor>
|           |       |   |       |                   \-- STRING('Hello')
|           |       |   |       \-- RPARENT
|           |       |   \-- SEMICOLON
|           |       \-- ENDSY
|           \-- SEMICOLON
|-- <compound-statement>
|   |-- BEGINSY
|   |-- <statement-list>
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(result)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- <procedure/function-call>
|   |   |                           |-- IDENT(Max)
|   |   |                           |-- LPARENT
|   |   |                           |-- <parameter-list>
|   |   |                           |   |-- <expression>
|   |   |                           |   |   \-- <simple-expression>
```

```
|   |   |                           |   |       \-- <term>
|   |   |                           |   |           \-- <factor>
|   |   |                           |   |               \-- INTCON(15)
|   |   |                           |   |-- COMMA
|   |   |                           |   \-- <expression>
|   |   |                           |       \-- <simple-expression>
|   |   |                           |           \-- <term>
|   |   |                           |               \-- <factor>
|   |   |                           |                   \-- INTCON(7)
|   |   |                           \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <procedure/function-call>
|   |   |       |-- IDENT(PrintNum)
|   |   |       |-- LPARENT
|   |   |       |-- <parameter-list>
|   |   |       |   \-- <expression>
|   |   |       |       \-- <simple-expression>
|   |   |       |           \-- <term>
|   |   |       |               \-- <factor>
|   |   |       |                   \-- IDENT(result)
|   |   |       \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <procedure/function-call>
|   |   |       |-- IDENT(PrintPair)
|   |   |       |-- LPARENT
|   |   |       |-- <parameter-list>
|   |   |       |   |-- <expression>
|   |   |       |   |   \-- <simple-expression>
|   |   |       |   |       \-- <term>
|   |   |       |   |           \-- <factor>
|   |   |       |   |               \-- INTCON(1)
|   |   |       |   |-- COMMA
|   |   |       |   |-- <expression>
|   |   |       |   |   \-- <simple-expression>
|   |   |       |   |       \-- <term>
|   |   |       |   |           \-- <factor>
|   |   |       |   |               \-- INTCON(2)
|   |   |       |   |-- COMMA
|   |   |       |   \-- <expression>
|   |   |       |       \-- <simple-expression>
|   |   |       |           \-- <term>
|   |   |       |               \-- <factor>
|   |   |       |                   \-- STRING('pasangan:')
|   |   |       \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <procedure/function-call>
|   |   |       |-- IDENT(Greet)
|   |   |       |-- LPARENT
|   |   |       \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <if-statement>
|   |   |       |-- IFSY
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               |-- NOTSY
|   |   |       |               \-- <factor>
|   |   |       |                   |-- LPARENT
|   |   |       |                   |-- <expression>
|   |   |       |                   |   |-- <simple-expression>
|   |   |       |                   |   |   \-- <term>
|   |   |       |                   |   |       \-- <factor>
|   |   |       |                   |   |           \-- IDENT(result)
|   |   |       |                   |   |-- <relational-operator>
|   |   |       |                   |   |   \-- GTR
|   |   |       |                   |   \-- <simple-expression>
|   |   |       |                   |       \-- <term>
|   |   |       |                   |           \-- <factor>
|   |   |       |                   |               \-- INTCON(100)
|   |   |       |                   \-- RPARENT
|   |   |       |-- THENSY
|   |   |       \-- <statement>
|   |   |           \-- <procedure/function-call>
|   |   |               |-- IDENT(writeln)
|   |   |               |-- LPARENT
```

```
    |   |   |               |-- <parameter-list>
    |   |   |               |   \-- <expression>
    |   |   |               |       \-- <simple-expression>
    |   |   |               |           \-- <term>
    |   |   |               |               \-- <factor>
    |   |   |               |                   \-- STRING('dalam range')
    |   |   |               \-- RPARENT
    |   |   |-- SEMICOLON
    |   |   \-- <statement>
    |   |       \-- <assignment-statement>
    |   |           |-- <variable>
    |   |           |   \-- IDENT(result)
    |   |           |-- BECOMES
    |   |           \-- <expression>
    |   |               \-- <simple-expression>
    |   |                   \-- <term>
    |   |                       \-- <factor>
    |   |                           \-- <procedure/function-call>
    |   |                               |-- IDENT(Max)
    |   |                               |-- LPARENT
    |   |                               |-- <parameter-list>
    |   |                               |   |-- <expression>
    |   |                               |   |   \-- <simple-expression>
    |   |                               |   |       \-- <term>
    |   |                               |   |           \-- <factor>
    |   |                               |   |               \-- <procedure/function-call>
    |   |                               |   |                   |-- IDENT(Max)
    |   |                               |   |                   |-- LPARENT
    |   |                               |   |                   |-- <parameter-list>
    |   |                               |   |                   |   |-- <expression>
    |   |                               |   |                   |   |   \-- <simple-expression>
    |   |                               |   |                   |   |       \-- <term>
    |   |                               |   |                   |   |           \-- <factor>
    |   |                               |   |                   |   |               \-- INTCON(1)
    |   |                               |   |                   |   |-- COMMA
    |   |                               |   |                   |   \-- <expression>
    |   |                               |   |                   |       \-- <simple-expression>
    |   |                               |   |                   |           \-- <term>
    |   |                               |   |                   |               \-- <factor>
    |   |                               |   |                   |                   \-- INTCON(2)
    |   |                               |   |                   \-- RPARENT
    |   |                               |   |-- COMMA
    |   |                               |   \-- <expression>
    |   |                               |       \-- <simple-expression>
    |   |                               |           \-- <term>
    |   |                               |               \-- <factor>
    |   |                               |                   \-- INTCON(3)
    |   |                               \-- RPARENT
    |   \-- ENDSY
    \-- PERIOD
```

Masukkan (Input)

```
program TestComprehensive;

const
  LIMIT == 50;
  BASE == 1;
  NAME == 'Arion';

type
  Status == (active, inactive, pending);
  Grade == 'A'..'E';
  Student == record
    name: string;
    score: integer;
  end;
  Scores == array[0..100] of integer;
```

```
var
  i, j, total: integer;
  avg: real;

function Abs(n: integer): integer;
begin
  if n < 0 then
    Abs := -n
  else
    Abs := n;
end;

function Clamp(val, lo, hi: integer): integer;
begin
  if val < lo then
    Clamp := lo
  else if val > hi then
    Clamp := hi
  else
    Clamp := val;
end;

procedure PrintStatus(n: integer; msg: string);
begin
  writeln(msg);
  writeln(n);
end;

begin
  total := 0;
  i := BASE;

  total := (10 + 20) * 3 - Abs(-5);
  avg := total / 6.0;
  i := total + 7;
  j := total - 3;

  if (i > 0) and (j >= 0) then
    PrintStatus(i, NAME)
  else if (i == 0) or (j < 0) then
    writeln('nol atau negatif')
  else
    writeln('kondisi lain');

  if not (i > LIMIT) then
    i := Clamp(i, 0, LIMIT);

  for i := 1 to 5 do
    writeln(i);

  for i := 5 downto 1 do
    writeln(i);

  while i < 10 do
    i := i + 1;
```

```
  repeat
    j := j - 1
  until j == 0;

  case i of
    0: writeln('nol');
    1, 2, 3: writeln('kecil');
    10: writeln('sepuluh')
  end;

  writeln(Clamp(100, 0, 50))
end.
```

Keluaran (output)

```
Input path file: test/milestone-2/text6.txt
Gagal membaca file
Input path file: test/milestone-2/test6.txt
\-- <program>
    |-- <program-header>
    |   |-- PROGRAMSY
    |   |-- IDENT(TestComprehensive)
    |   \-- SEMICOLON
    |-- <declaration-part>
    |   |-- <const-declaration>
    |   |   |-- CONSTSY
    |   |   |-- IDENT(LIMIT)
    |   |   |-- EQL
    |   |   |-- <constant>
    |   |   |   \-- INTCON(50)
    |   |   |-- SEMICOLON
    |   |   |-- IDENT(BASE)
    |   |   |-- EQL
    |   |   |-- <constant>
    |   |   |   \-- INTCON(1)
    |   |   |-- SEMICOLON
    |   |   |-- IDENT(NAME)
    |   |   |-- EQL
    |   |   |-- <constant>
    |   |   |   \-- STRING('Arion')
    |   |   \-- SEMICOLON
    |   |-- <type-declaration>
    |   |   |-- TYPESY
    |   |   |-- IDENT(Status)
    |   |   |-- EQL
    |   |   |-- <type>
    |   |   |   \-- <enumerated>
    |   |   |       |-- LPARENT
    |   |   |       |-- IDENT(active)
    |   |   |       |-- COMMA
    |   |   |       |-- IDENT(inactive)
    |   |   |       |-- COMMA
    |   |   |       |-- IDENT(pending)
    |   |   |       \-- RPARENT
    |   |   |-- SEMICOLON
    |   |   |-- IDENT(Grade)
    |   |   |-- EQL
    |   |   |-- <type>
    |   |   |   \-- <range>
    |   |   |       |-- <constant>
    |   |   |       |   \-- CHARCON('A')
    |   |   |       |-- PERIOD
    |   |   |       |-- PERIOD
```

```
|   |   |       \-- <constant>
|   |   |           \-- CHARCON('E')
|   |   |-- SEMICOLON
|   |   |-- IDENT(Student)
|   |   |-- EQL
|   |   |-- <type>
|   |   |   \-- <record-type>
|   |   |       |-- RECORDSY
|   |   |       |-- <field-list>
|   |   |       |   |-- <field-part>
|   |   |       |   |   |-- <identifier-list>
|   |   |       |   |   |   \-- IDENT(name)
|   |   |       |   |   |-- COLON
|   |   |       |   |   \-- <type>
|   |   |       |   |       \-- IDENT(string)
|   |   |       |   |-- SEMICOLON
|   |   |       |   |-- <field-part>
|   |   |       |   |   |-- <identifier-list>
|   |   |       |   |   |   \-- IDENT(score)
|   |   |       |   |   |-- COLON
|   |   |       |   |   \-- <type>
|   |   |       |   |       \-- IDENT(integer)
|   |   |       |   \-- SEMICOLON
|   |   |       \-- ENDSY
|   |   |-- SEMICOLON
|   |   |-- IDENT(Scores)
|   |   |-- EQL
|   |   |-- <type>
|   |   |   \-- <array-type>
|   |   |       |-- ARRAYSY
|   |   |       |-- LBRACK
|   |   |       |-- <range>
|   |   |       |   |-- <constant>
|   |   |       |   |   \-- INTCON(0)
|   |   |       |   |-- PERIOD
|   |   |       |   |-- PERIOD
|   |   |       |   \-- <constant>
|   |   |       |       \-- INTCON(100)
|   |   |       |-- RBRACK
|   |   |       |-- OFSY
|   |   |       \-- <type>
|   |   |           \-- IDENT(integer)
|   |   \-- SEMICOLON
|   |-- <var-declaration>
|   |   |-- VARSY
|   |   |-- <identifier-list>
|   |   |   |-- IDENT(i)
|   |   |   |-- COMMA
|   |   |   |-- IDENT(j)
|   |   |   |-- COMMA
|   |   |   \-- IDENT(total)
|   |   |-- COLON
|   |   |-- <type>
|   |   |   \-- IDENT(integer)
|   |   |-- SEMICOLON
|   |   |-- <identifier-list>
|   |   |   \-- IDENT(avg)
|   |   |-- COLON
|   |   |-- <type>
|   |   |   \-- IDENT(real)
|   |   \-- SEMICOLON
|   |-- <subprogram-declaration>
|   |   \-- <function-declaration>
|   |       |-- FUNCTIONSY
|   |       |-- IDENT(Abs)
|   |       |-- <formal-parameter-list>
|   |       |   |-- LPARENT
|   |       |   |-- <parameter-group>
|   |       |   |   |-- <identifier-list>
|   |       |   |   |   \-- IDENT(n)
```

```
|   |       |   |   |-- COLON
|   |       |   |   \-- IDENT(integer)
|   |       |   \-- RPARENT
|   |       |-- COLON
|   |       |-- IDENT(integer)
|   |       |-- SEMICOLON
|   |       |-- <block>
|   |       |   |-- <declaration-part>
|   |       |   \-- <compound-statement>
|   |       |       |-- BEGINSY
|   |       |       |-- <statement-list>
|   |       |       |   |-- <statement>
|   |       |       |   |   \-- <if-statement>
|   |       |       |   |       |-- IFSY
|   |       |       |   |       |-- <expression>
|   |       |       |   |       |   |-- <simple-expression>
|   |       |       |   |       |   |   \-- <term>
|   |       |       |   |       |   |       \-- <factor>
|   |       |       |   |       |   |           \-- IDENT(n)
|   |       |       |   |       |   |-- <relational-operator>
|   |       |       |   |       |   |   \-- LSS
|   |       |       |   |       |   \-- <simple-expression>
|   |       |       |   |       |       \-- <term>
|   |       |       |   |       |           \-- <factor>
|   |       |       |   |       |               \-- INTCON(0)
|   |       |       |   |       |-- THENSY
|   |       |       |   |       |-- <statement>
|   |       |       |   |       |   \-- <assignment-statement>
|   |       |       |   |       |       |-- <variable>
|   |       |       |   |       |       |   \-- IDENT(Abs)
|   |       |       |   |       |       |-- BECOMES
|   |       |       |   |       |       \-- <expression>
|   |       |       |   |       |           \-- <simple-expression>
|   |       |       |   |       |               |-- MINUS
|   |       |       |   |       |               \-- <term>
|   |       |       |   |       |                   \-- <factor>
|   |       |       |   |       |                       \-- IDENT(n)
|   |       |       |   |       |-- ELSESY
|   |       |       |   |       \-- <statement>
|   |       |       |   |           \-- <assignment-statement>
|   |       |       |   |               |-- <variable>
|   |       |       |   |               |   \-- IDENT(Abs)
|   |       |       |   |               |-- BECOMES
|   |       |       |   |               \-- <expression>
|   |       |       |   |                   \-- <simple-expression>
|   |       |       |   |                       \-- <term>
|   |       |       |   |                           \-- <factor>
|   |       |       |   |                               \-- IDENT(n)
|   |       |       |   \-- SEMICOLON
|   |       |       \-- ENDSY
|   |       \-- SEMICOLON
|   |-- <subprogram-declaration>
|   |   \-- <function-declaration>
|   |       |-- FUNCTIONSY
|   |       |-- IDENT(Clamp)
|   |       |-- <formal-parameter-list>
|   |       |   |-- LPARENT
|   |       |   |-- <parameter-group>
|   |       |   |   |-- <identifier-list>
|   |       |   |   |   |-- IDENT(val)
|   |       |   |   |   |-- COMMA
|   |       |   |   |   |-- IDENT(lo)
|   |       |   |   |   |-- COMMA
|   |       |   |   |   \-- IDENT(hi)
|   |       |   |   |-- COLON
|   |       |   |   \-- IDENT(integer)
|   |       |   \-- RPARENT
|   |       |-- COLON
|   |       |-- IDENT(integer)
|   |       |-- SEMICOLON
```

```
|   |       |-- <block>
|   |       |   |-- <declaration-part>
|   |       |   \-- <compound-statement>
|   |       |       |-- BEGINSY
|   |       |       |-- <statement-list>
|   |       |       |   |-- <statement>
|   |       |       |   |   \-- <if-statement>
|   |       |       |   |       |-- IFSY
|   |       |       |   |       |-- <expression>
|   |       |       |   |       |   |-- <simple-expression>
|   |       |       |   |       |   |   \-- <term>
|   |       |       |   |       |   |       \-- <factor>
|   |       |       |   |       |   |           \-- IDENT(val)
|   |       |       |   |       |   |-- <relational-operator>
|   |       |       |   |       |   |   \-- LSS
|   |       |       |   |       |   \-- <simple-expression>
|   |       |       |   |       |       \-- <term>
|   |       |       |   |       |           \-- <factor>
|   |       |       |   |       |               \-- IDENT(lo)
|   |       |       |   |       |-- THENSY
|   |       |       |   |       |-- <statement>
|   |       |       |   |       |   \-- <assignment-statement>
|   |       |       |   |       |       |-- <variable>
|   |       |       |   |       |       |   \-- IDENT(Clamp)
|   |       |       |   |       |       |-- BECOMES
|   |       |       |   |       |       \-- <expression>
|   |       |       |   |       |           \-- <simple-expression>
|   |       |       |   |       |               \-- <term>
|   |       |       |   |       |                   \-- <factor>
|   |       |       |   |       |                       \-- IDENT(lo)
|   |       |       |   |       |-- ELSESY
|   |       |       |   |       \-- <statement>
|   |       |       |   |           \-- <if-statement>
|   |       |       |   |               |-- IFSY
|   |       |       |   |               |-- <expression>
|   |       |       |   |               |   |-- <simple-expression>
|   |       |       |   |               |   |   \-- <term>
|   |       |       |   |               |   |       \-- <factor>
|   |       |       |   |               |   |           \-- IDENT(val)
|   |       |       |   |               |   |-- <relational-operator>
|   |       |       |   |               |   |   \-- GTR
|   |       |       |   |               |   \-- <simple-expression>
|   |       |       |   |               |       \-- <term>
|   |       |       |   |               |           \-- <factor>
|   |       |       |   |               |               \-- IDENT(hi)
|   |       |       |   |               |-- THENSY
|   |       |       |   |               |-- <statement>
|   |       |       |   |               |   \-- <assignment-statement>
|   |       |       |   |               |       |-- <variable>
|   |       |       |   |               |       |   \-- IDENT(Clamp)
|   |       |       |   |               |       |-- BECOMES
|   |       |       |   |               |       \-- <expression>
|   |       |       |   |               |           \-- <simple-expression>
|   |       |       |   |               |               \-- <term>
|   |       |       |   |               |                   \-- <factor>
|   |       |       |   |               |                       \-- IDENT(hi)
|   |       |       |   |               |-- ELSESY
|   |       |       |   |               \-- <statement>
|   |       |       |   |                   \-- <assignment-statement>
|   |       |       |   |                       |-- <variable>
|   |       |       |   |                       |   \-- IDENT(Clamp)
|   |       |       |   |                       |-- BECOMES
|   |       |       |   |                       \-- <expression>
|   |       |       |   |                           \-- <simple-expression>
|   |       |       |   |                               \-- <term>
|   |       |       |   |                                   \-- <factor>
|   |       |       |   |                                       \-- IDENT(val)
|   |       |       |   \-- SEMICOLON
|   |       |       \-- ENDSY
|   |       \-- SEMICOLON
```

```
|   \-- <subprogram-declaration>
|       \-- <procedure-declaration>
|           |-- PROCEDURESY
|           |-- IDENT(PrintStatus)
|           |-- <formal-parameter-list>
|           |   |-- LPARENT
|           |   |-- <parameter-group>
|           |   |   |-- <identifier-list>
|           |   |   |   \-- IDENT(n)
|           |   |   |-- COLON
|           |   |   \-- IDENT(integer)
|           |   |-- SEMICOLON
|           |   |-- <parameter-group>
|           |   |   |-- <identifier-list>
|           |   |   |   \-- IDENT(msg)
|           |   |   |-- COLON
|           |   |   \-- IDENT(string)
|           |   \-- RPARENT
|           |-- SEMICOLON
|           |-- <block>
|           |   |-- <declaration-part>
|           |   \-- <compound-statement>
|           |       |-- BEGINSY
|           |       |-- <statement-list>
|           |       |   |-- <statement>
|           |       |   |   \-- <procedure/function-call>
|           |       |   |       |-- IDENT(writeln)
|           |       |   |       |-- LPARENT
|           |       |   |       |-- <parameter-list>
|           |       |   |       |   \-- <expression>
|           |       |   |       |       \-- <simple-expression>
|           |       |   |       |           \-- <term>
|           |       |   |       |               \-- <factor>
|           |       |   |       |                   \-- IDENT(msg)
|           |       |   |       \-- RPARENT
|           |       |   |-- SEMICOLON
|           |       |   |-- <statement>
|           |       |   |   \-- <procedure/function-call>
|           |       |   |       |-- IDENT(writeln)
|           |       |   |       |-- LPARENT
|           |       |   |       |-- <parameter-list>
|           |       |   |       |   \-- <expression>
|           |       |   |       |       \-- <simple-expression>
|           |       |   |       |           \-- <term>
|           |       |   |       |               \-- <factor>
|           |       |   |       |                   \-- IDENT(n)
|           |       |   |       \-- RPARENT
|           |       |   \-- SEMICOLON
|           |       \-- ENDSY
|           \-- SEMICOLON
|-- <compound-statement>
|   |-- BEGINSY
|   |-- <statement-list>
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(total)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- INTCON(0)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(i)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
```

```
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- IDENT(BASE)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(total)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               |-- <term>
|   |   |               |   |-- <factor>
|   |   |               |   |   |-- LPARENT
|   |   |               |   |   |-- <expression>
|   |   |               |   |   |   \-- <simple-expression>
|   |   |               |   |   |       |-- <term>
|   |   |               |   |   |       |   \-- <factor>
|   |   |               |   |   |       |       \-- INTCON(10)
|   |   |               |   |   |       |-- <additive-operator>
|   |   |               |   |   |       |   \-- PLUS
|   |   |               |   |   |       \-- <term>
|   |   |               |   |   |           \-- <factor>
|   |   |               |   |   |               \-- INTCON(20)
|   |   |               |   |   \-- RPARENT
|   |   |               |   |-- <multiplicative-operator>
|   |   |               |   |   \-- TIMES
|   |   |               |   \-- <factor>
|   |   |               |       \-- INTCON(3)
|   |   |               |-- <additive-operator>
|   |   |               |   \-- MINUS
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- <procedure/function-call>
|   |   |                           |-- IDENT(Abs)
|   |   |                           |-- LPARENT
|   |   |                           |-- <parameter-list>
|   |   |                           |   \-- <expression>
|   |   |                           |       \-- <simple-expression>
|   |   |                           |           \-- <term>
|   |   |                           |               \-- <factor>
|   |   |                           |                   \-- INTCON(5)
|   |   |                           \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(avg)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   |-- <factor>
|   |   |                   |   \-- IDENT(total)
|   |   |                   |-- <multiplicative-operator>
|   |   |                   |   \-- RDIV
|   |   |                   \-- <factor>
|   |   |                       \-- REALCON(6.0)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(i)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               |-- <term>
|   |   |               |   \-- <factor>
|   |   |               |       \-- IDENT(total)
```

```
|   |   |               |-- <additive-operator>
|   |   |               |   \-- PLUS
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- INTCON(7)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <assignment-statement>
|   |   |       |-- <variable>
|   |   |       |   \-- IDENT(j)
|   |   |       |-- BECOMES
|   |   |       \-- <expression>
|   |   |           \-- <simple-expression>
|   |   |               |-- <term>
|   |   |               |   \-- <factor>
|   |   |               |       \-- IDENT(total)
|   |   |               |-- <additive-operator>
|   |   |               |   \-- MINUS
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- INTCON(3)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <if-statement>
|   |   |       |-- IFSY
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           |-- <factor>
|   |   |       |           |   |-- LPARENT
|   |   |       |           |   |-- <expression>
|   |   |       |           |   |   |-- <simple-expression>
|   |   |       |           |   |   |   \-- <term>
|   |   |       |           |   |   |       \-- <factor>
|   |   |       |           |   |   |           \-- IDENT(i)
|   |   |       |           |   |   |-- <relational-operator>
|   |   |       |           |   |   |   \-- GTR
|   |   |       |           |   |   \-- <simple-expression>
|   |   |       |           |   |       \-- <term>
|   |   |       |           |   |           \-- <factor>
|   |   |       |           |   |               \-- INTCON(0)
|   |   |       |           |   \-- RPARENT
|   |   |       |           |-- <multiplicative-operator>
|   |   |       |           |   \-- ANDSY
|   |   |       |           \-- <factor>
|   |   |       |               |-- LPARENT
|   |   |       |               |-- <expression>
|   |   |       |               |   |-- <simple-expression>
|   |   |       |               |   |   \-- <term>
|   |   |       |               |   |       \-- <factor>
|   |   |       |               |   |           \-- IDENT(j)
|   |   |       |               |   |-- <relational-operator>
|   |   |       |               |   |   \-- GEQ
|   |   |       |               |   \-- <simple-expression>
|   |   |       |               |       \-- <term>
|   |   |       |               |           \-- <factor>
|   |   |       |               |               \-- INTCON(0)
|   |   |       |               \-- RPARENT
|   |   |       |-- THENSY
|   |   |       |-- <statement>
|   |   |       |   \-- <procedure/function-call>
|   |   |       |       |-- IDENT(PrintStatus)
|   |   |       |       |-- LPARENT
|   |   |       |       |-- <parameter-list>
|   |   |       |       |   |-- <expression>
|   |   |       |       |   |   \-- <simple-expression>
|   |   |       |       |   |       \-- <term>
|   |   |       |       |   |           \-- <factor>
|   |   |       |       |   |               \-- IDENT(i)
|   |   |       |       |   |-- COMMA
```

```
|   |   |       |       |   \-- <expression>
|   |   |       |       |       \-- <simple-expression>
|   |   |       |       |           \-- <term>
|   |   |       |       |               \-- <factor>
|   |   |       |       |                   \-- IDENT(NAME)
|   |   |       |       \-- RPARENT
|   |   |       |-- ELSESY
|   |   |       \-- <statement>
|   |   |           \-- <if-statement>
|   |   |               |-- IFSY
|   |   |               |-- <expression>
|   |   |               |   \-- <simple-expression>
|   |   |               |       |-- <term>
|   |   |               |       |   \-- <factor>
|   |   |               |       |       |-- LPARENT
|   |   |               |       |       |-- <expression>
|   |   |               |       |       |   |-- <simple-expression>
|   |   |               |       |       |   |   \-- <term>
|   |   |               |       |       |   |       \-- <factor>
|   |   |               |       |       |   |           \-- IDENT(i)
|   |   |               |       |       |   |-- <relational-operator>
|   |   |               |       |       |   |   \-- EQL
|   |   |               |       |       |   \-- <simple-expression>
|   |   |               |       |       |       \-- <term>
|   |   |               |       |       |           \-- <factor>
|   |   |               |       |       |               \-- INTCON(0)
|   |   |               |       |       \-- RPARENT
|   |   |               |       |-- <additive-operator>
|   |   |               |       |   \-- ORSY
|   |   |               |       \-- <term>
|   |   |               |           \-- <factor>
|   |   |               |               |-- LPARENT
|   |   |               |               |-- <expression>
|   |   |               |               |   |-- <simple-expression>
|   |   |               |               |   |   \-- <term>
|   |   |               |               |   |       \-- <factor>
|   |   |               |               |   |           \-- IDENT(j)
|   |   |               |               |   |-- <relational-operator>
|   |   |               |               |   |   \-- LSS
|   |   |               |               |   \-- <simple-expression>
|   |   |               |               |       \-- <term>
|   |   |               |               |           \-- <factor>
|   |   |               |               |               \-- INTCON(0)
|   |   |               |               \-- RPARENT
|   |   |               |-- THENSY
|   |   |               |-- <statement>
|   |   |               |   \-- <procedure/function-call>
|   |   |               |       |-- IDENT(writeln)
|   |   |               |       |-- LPARENT
|   |   |               |       |-- <parameter-list>
|   |   |               |       |   \-- <expression>
|   |   |               |       |       \-- <simple-expression>
|   |   |               |       |           \-- <term>
|   |   |               |       |               \-- <factor>
|   |   |               |       |                   \-- STRING('nol atau negatif')
|   |   |               |       \-- RPARENT
|   |   |               |-- ELSESY
|   |   |               \-- <statement>
|   |   |                   \-- <procedure/function-call>
|   |   |                       |-- IDENT(writeln)
|   |   |                       |-- LPARENT
|   |   |                       |-- <parameter-list>
|   |   |                       |   \-- <expression>
|   |   |                       |       \-- <simple-expression>
|   |   |                       |           \-- <term>
|   |   |                       |               \-- <factor>
|   |   |                       |                   \-- STRING('kondisi lain')
|   |   |                       \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- <statement>
```

```
|   |   |   \-- <if-statement>
|   |   |       |-- IFSY
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               |-- NOTSY
|   |   |       |               \-- <factor>
|   |   |       |                   |-- LPARENT
|   |   |       |                   |-- <expression>
|   |   |       |                   |   |-- <simple-expression>
|   |   |       |                   |   |   \-- <term>
|   |   |       |                   |   |       \-- <factor>
|   |   |       |                   |   |           \-- IDENT(i)
|   |   |       |                   |   |-- <relational-operator>
|   |   |       |                   |   |   \-- GTR
|   |   |       |                   |   \-- <simple-expression>
|   |   |       |                   |       \-- <term>
|   |   |       |                   |           \-- <factor>
|   |   |       |                   |               \-- IDENT(LIMIT)
|   |   |       |                   \-- RPARENT
|   |   |       |-- THENSY
|   |   |       \-- <statement>
|   |   |           \-- <assignment-statement>
|   |   |               |-- <variable>
|   |   |               |   \-- IDENT(i)
|   |   |               |-- BECOMES
|   |   |               \-- <expression>
|   |   |                   \-- <simple-expression>
|   |   |                       \-- <term>
|   |   |                           \-- <factor>
|   |   |                               \-- <procedure/function-call>
|   |   |                                   |-- IDENT(Clamp)
|   |   |                                   |-- LPARENT
|   |   |                                   |-- <parameter-list>
|   |   |                                   |   |-- <expression>
|   |   |                                   |   |   \-- <simple-expression>
|   |   |                                   |   |       \-- <term>
|   |   |                                   |   |           \-- <factor>
|   |   |                                   |   |               \-- IDENT(i)
|   |   |                                   |   |-- COMMA
|   |   |                                   |   |-- <expression>
|   |   |                                   |   |   \-- <simple-expression>
|   |   |                                   |   |       \-- <term>
|   |   |                                   |   |           \-- <factor>
|   |   |                                   |   |               \-- INTCON(0)
|   |   |                                   |   |-- COMMA
|   |   |                                   |   \-- <expression>
|   |   |                                   |       \-- <simple-expression>
|   |   |                                   |           \-- <term>
|   |   |                                   |               \-- <factor>
|   |   |                                   |                   \-- IDENT(LIMIT)
|   |   |                                   \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <for-statement>
|   |   |       |-- FORSY
|   |   |       |-- IDENT(i)
|   |   |       |-- BECOMES
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               \-- INTCON(1)
|   |   |       |-- TOSY
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               \-- INTCON(5)
```

```
|   |   |       |-- DOSY
|   |   |       \-- <statement>
|   |   |           \-- <procedure/function-call>
|   |   |               |-- IDENT(writeln)
|   |   |               |-- LPARENT
|   |   |               |-- <parameter-list>
|   |   |               |   \-- <expression>
|   |   |               |       \-- <simple-expression>
|   |   |               |           \-- <term>
|   |   |               |               \-- <factor>
|   |   |               |                   \-- IDENT(i)
|   |   |               \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <for-statement>
|   |   |       |-- FORSY
|   |   |       |-- IDENT(i)
|   |   |       |-- BECOMES
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               \-- INTCON(5)
|   |   |       |-- DOWNTOSY
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               \-- INTCON(1)
|   |   |       |-- DOSY
|   |   |       \-- <statement>
|   |   |           \-- <procedure/function-call>
|   |   |               |-- IDENT(writeln)
|   |   |               |-- LPARENT
|   |   |               |-- <parameter-list>
|   |   |               |   \-- <expression>
|   |   |               |       \-- <simple-expression>
|   |   |               |           \-- <term>
|   |   |               |               \-- <factor>
|   |   |               |                   \-- IDENT(i)
|   |   |               \-- RPARENT
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <while-statement>
|   |   |       |-- WHILESY
|   |   |       |-- <expression>
|   |   |       |   |-- <simple-expression>
|   |   |       |   |   \-- <term>
|   |   |       |   |       \-- <factor>
|   |   |       |   |           \-- IDENT(i)
|   |   |       |   |-- <relational-operator>
|   |   |       |   |   \-- LSS
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               \-- INTCON(10)
|   |   |       |-- DOSY
|   |   |       \-- <statement>
|   |   |           \-- <assignment-statement>
|   |   |               |-- <variable>
|   |   |               |   \-- IDENT(i)
|   |   |               |-- BECOMES
|   |   |               \-- <expression>
|   |   |                   \-- <simple-expression>
|   |   |                       |-- <term>
|   |   |                       |   \-- <factor>
|   |   |                       |       \-- IDENT(i)
|   |   |                       |-- <additive-operator>
|   |   |                       |   \-- PLUS
|   |   |                       \-- <term>
```

```
|   |   |                           \-- <factor>
|   |   |                               \-- INTCON(1)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <repeat-statement>
|   |   |       |-- REPEATSY
|   |   |       |-- <statement-list>
|   |   |       |   \-- <statement>
|   |   |       |       \-- <assignment-statement>
|   |   |       |           |-- <variable>
|   |   |       |           |   \-- IDENT(j)
|   |   |       |           |-- BECOMES
|   |   |       |           \-- <expression>
|   |   |       |               \-- <simple-expression>
|   |   |       |                   |-- <term>
|   |   |       |                   |   \-- <factor>
|   |   |       |                   |       \-- IDENT(j)
|   |   |       |                   |-- <additive-operator>
|   |   |       |                   |   \-- MINUS
|   |   |       |                   \-- <term>
|   |   |       |                       \-- <factor>
|   |   |       |                           \-- INTCON(1)
|   |   |       |-- UNTILSY
|   |   |       \-- <expression>
|   |   |           |-- <simple-expression>
|   |   |           |   \-- <term>
|   |   |           |       \-- <factor>
|   |   |           |           \-- IDENT(j)
|   |   |           |-- <relational-operator>
|   |   |           |   \-- EQL
|   |   |           \-- <simple-expression>
|   |   |               \-- <term>
|   |   |                   \-- <factor>
|   |   |                       \-- INTCON(0)
|   |   |-- SEMICOLON
|   |   |-- <statement>
|   |   |   \-- <case-statement>
|   |   |       |-- CASESY
|   |   |       |-- <expression>
|   |   |       |   \-- <simple-expression>
|   |   |       |       \-- <term>
|   |   |       |           \-- <factor>
|   |   |       |               \-- IDENT(i)
|   |   |       |-- OFSY
|   |   |       |-- <case-block>
|   |   |       |   |-- <constant>
|   |   |       |   |   \-- INTCON(0)
|   |   |       |   |-- COLON
|   |   |       |   |-- <statement>
|   |   |       |   |   \-- <procedure/function-call>
|   |   |       |   |       |-- IDENT(writeln)
|   |   |       |   |       |-- LPARENT
|   |   |       |   |       |-- <parameter-list>
|   |   |       |   |       |   \-- <expression>
|   |   |       |   |       |       \-- <simple-expression>
|   |   |       |   |       |           \-- <term>
|   |   |       |   |       |               \-- <factor>
|   |   |       |   |       |                   \-- STRING('nol')
|   |   |       |   |       \-- RPARENT
|   |   |       |   |-- SEMICOLON
|   |   |       |   \-- <case-block>
|   |   |       |       |-- <constant>
|   |   |       |       |   \-- INTCON(1)
|   |   |       |       |-- COMMA
|   |   |       |       |-- <constant>
|   |   |       |       |   \-- INTCON(2)
|   |   |       |       |-- COMMA
|   |   |       |       |-- <constant>
|   |   |       |       |   \-- INTCON(3)
|   |   |       |       |-- COLON
```

```
|   |   |       |       |-- <statement>
|   |   |       |       |   \-- <procedure/function-call>
|   |   |       |       |       |-- IDENT(writeln)
|   |   |       |       |       |-- LPARENT
|   |   |       |       |       |-- <parameter-list>
|   |   |       |       |       |   \-- <expression>
|   |   |       |       |       |       \-- <simple-expression>
|   |   |       |       |       |           \-- <term>
|   |   |       |       |       |               \-- <factor>
|   |   |       |       |       |                   \-- STRING('kecil')
|   |   |       |       |       \-- RPARENT
|   |   |       |       |-- SEMICOLON
|   |   |       |       \-- <case-block>
|   |   |       |           |-- <constant>
|   |   |       |           |   \-- INTCON(10)
|   |   |       |           |-- COLON
|   |   |       |           \-- <statement>
|   |   |       |               \-- <procedure/function-call>
|   |   |       |                   |-- IDENT(writeln)
|   |   |       |                   |-- LPARENT
|   |   |       |                   |-- <parameter-list>
|   |   |       |                   |   \-- <expression>
|   |   |       |                   |       \-- <simple-expression>
|   |   |       |                   |           \-- <term>
|   |   |       |                   |               \-- <factor>
|   |   |       |                   |                   \-- STRING('sepuluh')
|   |   |       |                   \-- RPARENT
|   |   |       \-- ENDSY
|   |   |-- SEMICOLON
|   |   \-- <statement>
|   |       \-- <procedure/function-call>
|   |           |-- IDENT(writeln)
|   |           |-- LPARENT
|   |           |-- <parameter-list>
|   |           |   \-- <expression>
|   |           |       \-- <simple-expression>
|   |           |           \-- <term>
|   |           |               \-- <factor>
|   |           |                   \-- <procedure/function-call>
|   |           |                       |-- IDENT(Clamp)
|   |           |                       |-- LPARENT
|   |           |                       |-- <parameter-list>
|   |           |                       |   |-- <expression>
|   |           |                       |   |   \-- <simple-expression>
|   |           |                       |   |       \-- <term>
|   |           |                       |   |           \-- <factor>
|   |           |                       |   |               \-- INTCON(100)
|   |           |                       |   |-- COMMA
|   |           |                       |   |-- <expression>
|   |           |                       |   |   \-- <simple-expression>
|   |           |                       |   |       \-- <term>
|   |           |                       |   |           \-- <factor>
|   |           |                       |   |               \-- INTCON(0)
|   |           |                       |   |-- COMMA
|   |           |                       |   \-- <expression>
|   |           |                       |       \-- <simple-expression>
|   |           |                       |           \-- <term>
|   |           |                       |               \-- <factor>
|   |           |                       |                   \-- INTCON(50)
|   |           |                       \-- RPARENT
|   |           \-- RPARENT
|   \-- ENDSY
\-- PERIOD
```

BAB IV ATURAN GRAMMAR

```
/* 1. Program Structure */
<program> ::=
    <program-header>
    <declaration-part>
    <compound-statement>
    period

<program-header> ::=
    programsy
    ident
    semicolon

<declaration-part> ::=
    { <const-declaration> }
    { <type-declaration> }
    { <var-declaration> }
    { <subprogram-declaration> }


/* 2. Declaration */
<const-declaration> ::=
    constsy
    ident eql <constant> semicolon
    { ident eql <constant> semicolon }
```

```
<constant> ::=
    charcon
    | string
    | [ plus | minus ]
      ( ident | intcon | realcon )

<type-declaration> ::=
    typesy
    ident eql <type> semicolon
    { ident eql <type> semicolon }

<var-declaration> ::=
    varsy
    <identifier-list> colon <type> semicolon
    { <identifier-list> colon <type> semicolon }

<identifier-list> ::=
    ident
    { comma ident }


/* 3. Type */
<type> ::=
    ident
    | <array-type>
    | <range>
    | <enumerated>
    | <record-type>

<array-type> ::=
    arraysy
```

```
    lbrack
    ( <range> | ident )
    rbrack
    ofsy
    <type>

<range> ::=
    <constant>
    period
    period
    <constant>

<enumerated> ::=
    lparent
    ident
    { comma ident }
    rparent

<record-type> ::=
    recordsy
    <field-list>
    endsy

<field-list> ::=
    <field-part>
    { semicolon <field-part> }

<field-part> ::=
    <identifier-list>
    colon
    <type>
```

```
/* 4. Subprogram */
<subprogram-declaration> ::=
    <procedure-declaration>
    | <function-declaration>

<procedure-declaration> ::=
    proceduresy
    ident
    [ <formal-parameter-list> ]
    semicolon
    <block>
    semicolon

<function-declaration> ::=
    functionsy
    ident
    [ <formal-parameter-list> ]
    colon
    ident
    semicolon
    <block>
    semicolon

<block> ::=
    <declaration-part>
    <compound-statement>

<formal-parameter-list> ::=
    lparent
```

```
    <parameter-group>
    { semicolon <parameter-group> }
    rparent

<parameter-group> ::=
    <identifier-list>
    colon
    ( ident | <array-type> )


/* 5. Statement */
<compound-statement> ::=
    beginsy
    <statement-list>
    endsy

<statement-list> ::=
    <statement>
    { semicolon <statement> }

<statement> ::=
    [ <assignment-statement>
    | <if-statement>
    | <case-statement>
    | <while-statement>
    | <repeat-statement>
    | <for-statement>
    | <procedure/function-call> ]

<variable> ::=
    ident
```

```
    { <component-variable> }

<component-variable> ::=
    ( lbrack <index-list> rbrack )
    | ( period ident )

<index-list> ::=
    ( intcon | charcon | ident )
    { comma <index-list> }

<assignment-statement> ::=
    <variable>
    becomes
    <expression>

<if-statement> ::=
    ifsy
    <expression>
    thensy
    <statement>
    [ elsesy <statement> ]

<case-statement> ::=
    casesy
    <expression>
    ofsy
    <case-block>
    endsy

<case-block> ::=
    <constant>
```

```
    { comma <constant> }
    colon
    <statement>
    { semicolon [ <case-block> ] }

<while-statement> ::=
    whilesy
    <expression>
    dosy
    <statement>

<repeat-statement> ::=
    repeatsy
    <statement-list>
    untilsy
    <expression>

<for-statement> ::=
    forsy
    ident
    becomes
    <expression>
    ( tosy | downtosy )
    <expression>
    dosy
    <statement>


/* 6. Procedure / Function Call */
<procedure/function-call> ::=
    ident
```

```
    lparent
    [ <parameter-list> ]
    rparent

<parameter-list> ::=
    <expression>
    { comma <expression> }


/* 7. Expression */
<expression> ::=
    <simple-expression>
    [ <relational-operator> <simple-expression> ]

<simple-expression> ::=
    [ plus | minus ]
    <term>
    { <additive-operator> <term> }

<term> ::=
    <factor>
    { <multiplicative-operator> <factor> }

<factor> ::=
    ident
    | intcon
    | realcon
    | charcon
    | string
    | ( lparent <expression> rparent )
    | ( notsy <factor> )
```

```
    | <procedure/function-call>
    | <variable>

<relational-operator> ::=
    eql
    | neq
    | gtr
    | geq
    | lss
    | leq

<additive-operator> ::=
    plus
    | minus
    | orsy

<multiplicative-operator> ::=
    times
    | rdiv
    | idiv
    | imod
    | andsy
```

BAB V KESIMPULAN & SARAN

5.1 Kesimpulan

Pada tugas ini telah berhasil dirancang dan diimplementasikan sebuah syntax analyzer (parser) untuk bahasa Arion menggunakan metode Recursive Descent berdasarkan grammar yang telah di-hardcode sesuai spesifikasi. Program mampu melanjutkan hasil lexical analyzer dari milestone sebelumnya dengan memproses token menjadi struktur Parse Tree yang merepresentasikan hierarki sintaks program secara lengkap. Implementasi yang dilakukan juga telah memenuhi kebutuhan validasi sintaks, penanganan syntax error secara informatif, serta menghasilkan output parse tree baik pada terminal maupun file .txt, sehingga sistem yang dibangun dapat berfungsi sebagai fondasi utama untuk tahap semantic analysis pada pengembangan compiler Arion di milestone berikutnya.

5.2 Saran

Untuk pengembangan selanjutnya, program yang telah dibuat dapat terus disempurnakan dari segi performa, struktur, dan kemudahan penggunaan agar semakin optimal.

LAMPIRAN

Link Repository Github
https://github.com/Faizalfadaa/JKW-Tubes-IF2224-2026

Pembagian Tugas

NIM	Kontribusi
13524007	25%
13524039	25%
13524069	25%
13524097	25%

REFERENSI

"Let's Build A Simple Interpreter. Part 7: Abstract Syntax Trees"
https://ruslanspivak.com/lsbasi-part7

"Contoh Gambar Parse Trees"
https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png

"EBNF: A Notation to Describe Syntax"
https://ics.uci.edu/~pattis/ICS-33/lectures/ebnf.pdf